Foundations of Machine Learning

Universität Stuttgart

Sommersemester 2026

Inhaltsverzeichnis

1 Preface **7**

2 Ultra-Mega-basics **7**

3 Organizational **7**
3.1 How to get the Schein . . . 7

4 Introduction **8**
4.1 What is Machine Learning? . . . 8
4.2 What Does ML Learn? . . . 8
4.2.1 Regression . . . 8
4.2.2 Classification . . . 8
4.2.3 Supervised Learning . . . 8
4.2.4 Reinforcement Learning . . . 8
4.2.5 Unsupervised Learning . . . 9
Data Clustering . . . 9

5 Terminological Chaos **9**

6 Knowledge Discovery **9**
6.1 Problem of Knowledge Discovery . . . 9
6.2 Stages of Knowledge Discovery . . . 9

7 The Crisp-DM model **10**
7.1 Business Understanding . . . 10
7.2 Data Understanding . . . 10
Statistical Descriptions . . . 10
Quartiles and Quantiles . . . 11
7.3 Data Preparation . . . 11
7.3.1 Categorical data to numeric: 1-hot encoding . . . 11
7.3.2 Categorical data to numeric: Target encoding . . . 11
7.3.3 Numerical data to nominal data . . . 11
7.4 Modeling . . . 12
7.4.1 Select Modeling Technique . . . 12
7.5 Evaluation . . . 12
7.6 Deployment . . . 12

8 Dimensionality Reduction **12**
Mathematics . . . 12
Principle of parsimony . . . 12
8.1 How to reduce dimensionality . . . 12
8.1.1 (Semi)Formal definition of dimensionality reduction . . . 13
8.2 Dimensionality Reduction techniques . . . 13
8.2.1 Linear Transformations Refresher . . . 13
8.2.2 Sample Covariance Matrix Refresher . . . 13
8.2.3 Principal Component Analysis (PCA) . . . 14
Idea of PCA: . . . 14
Finding the best d axes (PCA) . . . 14
Finding the first axis . . . 15
Optimization via Lagrange multipliers . . . 15
Induction step (more axes) . . . 16
Final interpretation . . . 16
Intuition . . . 16

9 Cluster Analysis **16**
9.1 Goal of clustering . . . 16
9.2 Clustering Task . . . 16
9.3 Cardinality/Number of clusters . . . 16
9.4 Evaluation . . . 16
9.4.1 Intrinsic . . . 17
Dunn Index . . . 17
Silhouette Coefficient . . . 17
9.4.2 Extrinsic Metrics . . . 17
Purity . . . 18
Mutual Information . . . 18
Rand Index . . . 18
Interpretation . . . 18
9.5 Loss Function vs Evaluation Metric . . . 19
9.6 K-Means . . . 19
9.6.1 Evaluate Quality . . . 19
9.6.2 Variations . . . 20
9.6.3 Choice of K . . . 20
9.6.4 Problematic Configurations . . . 20

10 Expectation Maximization **20**
Parameters: . . . 21
Refresher on probabilistic distributions: . . . 21
10.1 Gaussian Models of Individual Distributions . . . 21
10.1.1 Univariate Normal Distribution . . . 21
10.1.2 Multivariate Normal Distribution . . . 21
10.2 Model Estimation . . . 21
Maximum Likelihood Estimation (MLE): . . . 21
10.3 Cluster Assignment . . . 22
Assumption — Independently and Identically Distributed (i.i.d.): . . . 22
10.3.1 Problem: Initialization . . . 22
10.3.2 Example . . . 22

11 Linear Regression **23**
11.1 Interpolation . . . 23
11.1.1 Lagrange Interpolation . . . 23
11.1.2 Piecewise Linear Interpolation . . . 23
11.1.3 Spline Interpolation . . . 23
11.2 Regression . . . 24
11.2.1 Variants . . . 24
11.2.2 Multiple Linear Regression . . . 24
Scalar form: . . . 24
Vector form: . . . 24
Matrix form: . . . 25
Optimization objective: . . . 25
Determine Extreme Value . . . 25
Beyond Linear Input . . . 25
Example: Polynomial features . . . 25
Example: Piece-wise features (in 1D) . . . 26
Example: Radial Basis Function (RBF) . . . 26
11.3 Model Quality of Linear Regression . . . 26
11.3.1 Mean vs Median . . . 26
11.3.2 The Hat Matrix . . . 27
11.3.3 Residuals . . . 27
11.4 Investigating Model Assumptions . . . 28
11.5 Goals of Linear Regression Analysis . . . 28
Estimator Variance . . . 28
Overfitting and generalisation . . . 28
Bias–Variance decomposition . . . 28
11.6 Reducing Model Complexity . . . 29
11.6.1 Subset Selection . . . 29
11.6.2 Ridge Regression (L2) . . . 30
11.6.3 Lasso Regression (L1) . . . 30
11.6.4 Choosing hyperparameter λ . . . 30
k-fold cross validation . . . 31

12 Classification **31**
12.1 Ground Truth Classifier . . . 31
12.2 Defining the Task of Learning a Classifier . . . 31
12.2.1 Empirical Risk Minimization (ERM) . . . 31

13 Validation and Evaluation **32**
13.1 Combating Overfitting . . . 32
13.2 Evaluation Metrics for Classifiers . . . 32
13.2.1 Global Metric: Accuracy . . . 33
13.2.2 Classification Errors . . . 33
13.3 Data Leakage . . . 33

14 Naïve Bayes **33**
14.1 Bayes Classifier . . . 33
14.2 Full (Non-Naïve) Bayes Classifier . . . 34
14.3 Naïve Bayes Classifier . . . 34
14.3.1 Smoothing for Naïve Bayes Classifier . . . 35
Laplace Smoothing . . . 35
Generalized Additive Smoothing (Lidstone) . . . 35
Implementation Detail (Log-Transformation): . . . 35
14.3.2 Handling Numerical (Continuous) Values . . . 35

15 Logistic Regression **37**
15.1 Bayes Error & Expected Prediction Error . . . 37
15.1.1 Expected Prediction Error for Classification Task . . . 37
15.1.2 Optimal Bayes Classifier . . . 38
15.2 Classification by Logistic Regression . . . 38
15.2.1 The Logit Transformation . . . 38
1. Probability vs. Odds: . . . 38
2. The Logit (Log-Odds): . . . 38
Why do this conversion? . . . 39
15.2.2 Decision Boundary . . . 39
15.3 Logistic Regression: Multi-Class Case . . . 39
15.3.1 Likelihood Formulation . . . 40
15.4 Maximum Log-Likelihood Optimization . . . 40
15.4.1 Generalization via Cross-Entropy Loss . . . 40
15.5 Two-Class Log-Likelihood Optimization Analysis . . . 41
15.5.1 Numerical Optimization via the Newton-Raphson Method . . . 41

16 Decision Trees **42**
16.1 Measuring Distributions . . . 42
16.1.1 Entropy . . . 42
Entropy of coin tosses . . . 42
Cross Entropy . . . 42
Relative entropy (Kullback-Leibler Divergence) . . . 42
16.2 Decision Trees . . . 43
16.2.1 Inductive Construction of Decision Trees . . . 43
Decision tree induction . . . 44
Avoid overfitting . . . 44
Overfitting . . . 45
Estimating generalization error . . . 45
Tree pruning . . . 45
16.2.2 Attribute Splitting . . . 45
16.2.3 Pros and Cons of decision trees . . . 45

17 Bagging and Boosting **46**
17.1 Constructing a Random Forest . . . 46
17.1.1 Reducing Variance . . . 46
17.1.2 Bagging Mechanism in Random Forests . . . 47
Using Random Forest for Classification: . . . 47
Generalization Error of an Ensemble of Decision Trees: . . . 47
Advantages: . . . 47
17.1.3 Interpreting Random Forests . . . 47
Attribute Importance Score: Mean Decrease of Accuracy (MDA) . . . 47
Attribute Importance Score: Mean Decrease of Impurity (MDI) . . . 48
17.1.4 Computational complexity . . . 48

18 Explainable Machine Learning **48**
18.1 Local Feature Attributions by Shapley Values . . . 48
18.1.1 Properties of Shapley Values . . . 49

19 Decision Stumps **49**
19.1 Selecting the Best Decision Stump . . . 49

20 AdaBoost **49**
20.1 AdaBoost Scheme . . . 49
20.2 Discrete AdaBoost Algorithm . . . 50
20.3 Understanding AdaBoost via Forward Stagewise Additive Modeling . . . 50
20.3.1 Forward Stagewise Additive Modeling Algorithm . . . 50
Connection to AdaBoost: . . . 51

21 Understanding losses **51**
21.1 Comparing Loss functions for binary classification . . . 51
21.2 Comparing loss functions for regression . . . 52
21.3 Takeaway . . . 52

22 Support Vector Machines **52**
22.1 Perceptron Algorithm . . . 52
22.1.1 Linear Separability . . . 52
22.1.2 The perceptron classifier . . . 53
22.2 Support Vectors . . . 54
22.2.1 SVM Optimization Problem . . . 54
22.3 Soft margin and Hinge Loss . . . 55
22.3.1 Introducing 'slack' variables . . . 55
22.3.2 Soft margin solution . . . 55
22.3.3 Loss function . . . 56
22.4 Gradient Descent over convex function . . . 56
22.4.1 Optimization . . . 56
22.4.2 Convex functions . . . 57
22.4.3 Gradient Descent algorithm for SVM . . . 57
Sub-gradient for Hinge loss . . . 57
Sub-gradient descent algorithm for SVM . . . 58
22.5 The dual problem . . . 58
22.5.1 Primal vs dual problem . . . 58
22.5.2 Revisit optimization problem for hard margin case . . . 58
Lagrange multiplier (mental excursion) . . . 58
Algorithm for optimization with Lagrange Multiplier . . . 59
Lagrangian primal problem . . . 59
Finding the optimum . . . 59
22.5.3 Wolfe-dual Problem . . . 60
22.5.4 Summary: Primal and dual formulations . . . 60
22.6 Kernelization Tricks in SVMs . . . 60
22.6.1 Feature engineering using $\phi(x)$ . . . 60
22.6.2 Dual classifier in transformed feature space . . . 61
Kernels . . . 61
22.6.3 Dual classifier using kernels . . . 61
22.7 Kernels . . . 62
22.7.1 Properties . . . 62
22.7.2 Examples . . . 62
22.8 Transductive Classification . . . 62

23 Neural Networks **62**
23.1 Biological vs Artificial . . . 62
23.2 Reminder: Logistic Regression . . . 63
23.2.1 Logistic regression for binary cases . . . 63
Problem: Learning XOR with logistic regression . . . 64
23.3 Feed-Forward Networks and Activation Functions . . . 64
23.3.1 Network: Composing Functions . . . 64

Composition of linear functions for XOR Problem? . . . 64
23.3.2 One Layer in a Feed-Forward Network . . . 64
Activation functions . . . 65
23.3.3 Universal Approximation Theorem . . . 65
23.4 Backpropagation . . . 65
23.4.1 Computation Graphs: Language for representing the backpropagation problem 66
Notational Conventions . . . 66
Forward propagation in computational graph . . . 66
23.4.2 Backpropagation . . . 67
Example in a tiny computational graph . . . 67
23.5 Modelling a Feed-Forward Network predicting a Bernoulli distribution . . . 68
23.5.1 Loss function with Maximum Likelihood . . . 68
23.5.2 Probabilistic Model and Activation . . . 68
23.5.3 Negative Log-Likelihood Formulation . . . 68
23.5.4 Softplus function ζ . . . 69
23.5.5 Summary: Learning Bernoulli Output . . . 69
23.5.6 Softmax vs Softplus . . . 69
23.6 Modelling a Feed-Forward Network predicting a Multinoulli distribution . . . 69
23.6.1 Multinoulli Loss Function (Cross-Entropy) . . . 70
23.7 Modeling a Mixture Density distribution . . . 70
23.8 Optimizing Gradient Descent . . . 71
23.8.1 Initialization . . . 71
23.8.2 Optimization . . . 71
Newton's method in optimization (1D) . . . 71
Illustrating and Computing the Hessian . . . 72
Convergence . . . 72
23.8.3 Stochastic Gradient Descent . . . 72
Naive Stochastic Gradient Descent . . . 73
Adding Momentum . . . 73
23.9 ADAM - Adaptive Moment Estimation . . . 73
23.10 Regularization in DNNs: Lasso, Ridge and Constraints . . . 73
23.10.1 Paramer Norm Penalty . . . 73
23.10.2 L^2 Parameter Regularization . . . 73
23.10.3 L_1 and L_2 regularization by weight decay . . . 73
23.10.4 Column-Wise (Matrix) Regularization . . . 74
23.10.5 Regularization in DNNs: Data Augmentation . . . 74
Label Smoothing: Injecting noise at output target . . . 74
Injecting noise into the weights . . . 74
23.10.6 Multi-task learning . . . 74
23.10.7 Early stopping . . . 74
23.10.8 Adversarial Examples . . . 75
23.10.9 Grokking . . . 75
23.11 Regularization in DNNs: Dropout . . . 76
23.12 Convolutional Neural Networks . . . 76
23.12.1 Reduce Dimensionality: Subsampling by pooling . . . 76
23.13 Recurrent Neural Networks . . . 77
23.14 When to use DNNs . . . 77
23.14.1 Curse of Dimensionality . . . 77

1 Preface

These notes are mainly written in English, but there may be a few words in German and/or Spanish, since English is not my native language.

These notes contain all topics I valued as important for the exam, but some stuff that may have been trivial for me may not appear in the notes. So if anybody is using my notes to prepare for the exam, please be wary of this.

2 Ultra-Mega-basics

$$(AB)^T = B^T A^T$$

Positive definite matrix:

$$x^T Ax > 0 \forall x \neq 0$$

Regular matrix

$$A^{-1} \text{ exists}$$

Vector derivatives Let $x \in \mathbb{R}^n$. What is $\frac{\partial}{\partial x} x$?

Solution: $\frac{\partial}{\partial x} x = (\frac{\partial x_i}{\partial x_j})_{i=1,\dots,n;j=1,\dots,n}$. So its an $n \times n$ matrix of all combinations of the derivatives.

Derivative of a scalar in respect to a vector is now a row vector.

$$\frac{\partial}{\partial x} x^T \cdot x = [\frac{\partial}{\partial x_i} x_1^2 + \cdots + x_n^2]_{1 \leq i \leq n} = [2x_i]_{1 \leq i \leq n}$$

NOTE: This a standard, not the universal truth

We define $r_i := y_i - \hat{y}_i$

Mean Squared Error

$$MSE(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^{n} r_i^2$$

Mean Absolute Error

$$MAE(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^{n} |r_i|$$

Recommended Linear Algebra videos MIT by some guy dont remember the name

$\propto$ Means proportional to.

$D = \{x_1, \dots, x_n\} \subseteq X_1 \times \cdots \times X_m$ means that the dataset has n objects/observations, each of them having m attributes. X_m represents the set of values that attribute can take.

3 Organizational

Exam is in theory on August 13.

It is recommended to have attended before 'Foundations of Artificial Intelligence'

Coding in Python

3.1 How to get the Schein

- 11 Assignments, submission of three members
- Exam admission on an individual basis
- Present solutions twice (once in first 5 excersises, once in remaining sessions)
- Just have to give in excersises, don't seem that they are corrected.

4 Introduction

4.1 What is Machine Learning?

A computer is said to learn:

- from experience E
- with respect to a task T
- and a performance measure P

if its performance on T, measured by P, improves with experience E.

4.2 What Does ML Learn?

4.2.1 Regression

Given a set of existing values, predict a continuous output.
Example: Predicting daily temperature.
Experience E is typically a set of tuples (x, y), where:

- x: input (e.g., date)
- y: output (e.g., temperature)

Goal: find a function

$$\hat{f}(x) \approx y$$

This prediction process is called **inference**.
Output: continuous values

4.2.2 Classification

Assign input values to discrete classes.
Example: Determining whether a dog in an image is dangerous or not.
Experience consists of pairs (input, class label).
The prediction is also called **inference**.
Output: discrete values

4.2.3 Supervised Learning

Assume there exists a ground truth function:

$$f : X \to Y$$

Given training data D, the goal is to find $\hat{f}$ such that it approximates f as closely as possible (measured using P).

4.2.4 Reinforcement Learning

Starts with no prior experience, but with a policy (strategy).

1. Choose an action based on the current policy
2. Observe the outcome and receive a reward
3. Update the policy based on the reward
4. Repeat

4.2.5 Unsupervised Learning

No target values and no explicit rewards – only raw data.

Data Clustering Data points are grouped into clusters based on similarity.
Idea:

- Points in the same cluster are more similar to each other
- Points in different clusters are less similar

Used to uncover hidden patterns in data.

5 Terminological Chaos

Warning: There is quite a bit of inconsistent terminology in this field.
(See lecture slides for a more detailed breakdown.)

- Pattern recognition $\approx$ supervised learning
- Data mining $\neq$ strict synonym for knowledge discovery

6 Knowledge Discovery

The non-trivial process of identifying valid, novel, and useful patterns in data.

6.1 Problem of Knowledge Discovery

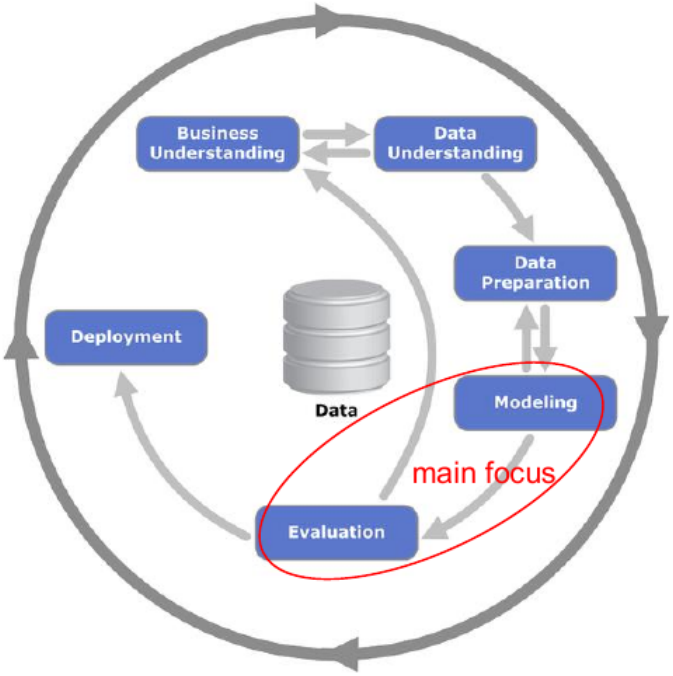
Given input data D, find a pattern S such that:

- S is an expression in an understandable language L describing correlations in D, and likely describve a new data D^{new}
- S is simpler than D

6.2 Stages of Knowledge Discovery

1. Data selection (choose relevant data)
2. Data cleaning
3. Data transformation (convert into usable format)
4. Data mining (search for patterns)
5. Evaluation (assess discovered patterns)
6. Knowledge representation (present results to users)

7 The Crisp-DM model



7.1 Business Understanding

1. Determine Business objectives: define success criteria.
2. Assess Situation (Risks, Resorces, Costs, Benefits...)
3. Determine Data Mining Goals
4. Produce Project Plan

7.2 Data Understanding

- Collect initial data: Identify relevant attributes, inconsistencies.
- Describe data: Statistical models for the data (mean, median...). Correlation between attributes.
- Explore Data
- Verify Data Quality: Errors in the data, missing values, false encodings...

Statistical Descriptions

- **Mode:** most frequent value. Always applicable
- **Median:** $\tilde{x}$ middle value. Only applicable to ordinal, interval and ratio scales.
- **Mean:** $\mu = \mathbb{E}_{x \in X}[x] = \frac{1}{n} \sum_{i=1}^{n} x_i$. Applicable to interval and ratio scales.
- **Standard deviation:** $\sigma^2 = \mathbb{E}_{x \in X}[(x - \mu)^2] = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \mu)^2$
- **Fisher-Pearson skewness:** $\mathbb{E}_{x \in X}[(\frac{x-\mu}{\sigma})^3]$

Quartiles and Quantiles 1st, 2nd, 3rd, 4th Quartile

$$IQR = Q_3 - Q_1$$

Outliers are values beyond **Tukey's fence**

$$[Q_1 - k \cdot IQR, Q_3 + k \cdot IQR]$$

for $k = 1.5$ or $k = 3$

7.3 Data Preparation

- Select data (feature selection): only get useful attributes. Discard highly correlated attributes
- Integrate data: Mix data from different sources. Careful with syntax/semantics/units of measurements.
- Clean Data: Correct false values, remove outliers, estimate missing values
- Format Data
- Feature Engineering: Define features from attribtes (e.g convert coordinates to distance). Normalize/transform single attributes.

7.3.1 Categorical data to numeric: 1-hot encoding

Given dataset $D = \{x_1, \dots, x_n\} \subseteq X_1 \times \cdots \times X_m$

Categorical attribute X_j with categories $dom(X_j) = \{a_1, \dots, a_k\}$

Define $\hat{X_j} \subseteq \{0,1\}^k$ and

$$\hat{x_{i,j,l}} = \begin{cases} 1 & j = l \\ 0 & else \end{cases}$$

Imagine it as a 3-D table with ones and zeroes.

Struggles with scalability and sparcity issues.

7.3.2 Categorical data to numeric: Target encoding

Categorical attribute A with categories $dom(A) = \{a_1, a_2, \dots, a_k\}$

Given target attribute Y with $dom(Y) = \{0,1\}$

Given dataset $D \subseteq A \times X \times Y$

$$\hat{a_i} = \frac{|\{a_i, ., 1\} \in D|}{|\{a_i, ., .\} \in D|} \in [0,1]$$

$$\hat{a}^{(i)} = \frac{\text{N}^{\text{o}} \text{ times category } a^{(i)} \text{ appears with } Y = 1}{\text{N}^{\text{o}} \text{ times category } a^{(i)} \text{ appears}}$$

Replace occurrances of a_i with $\hat{a_i}$

Struggles with overfitting and bias

7.3.3 Numerical data to nominal data

Given dataset $D = \{(x_1, y_1), \dots, (x_n, y_n)\} \subseteq X \times Y$

Given $X = \mathbb{R}$

Define $\hat{X} = \{1, \dots, k\}, k \geq 2$

Problem: Define $\hat{D}$ to represent original learning problem well. So we want to be able to group the values into k **bins**.

For $k = 2$ take median to decide between bin 1 or 2

7.4 Modeling

7.4.1 Select Modeling Technique

Take into account experience, and requirements. Generate a **Test Design** with data sets for **training data**, **validation data** and **test data**

Build and assess model

7.5 Evaluation

Define evaluation criteria during business understanding.

- Evaluate results: Findings of the project. **Algorithm must not determine business evaluation**
- Review process: Identify failures
- Determine next steps

7.6 Deployment

- Plan deployment: target platform, mobile, sensor quality...
- Plan Monitoring and Maintenance
- Produce Final Report
- Review Project

8 Dimensionality Reduction

Thought experiment: See around in your room, then close your eyes, and try to predict what will happen. Your prediction will of course not be a 1:1 representation of the room to the pixel level. Your predictions are semantic, where that guy is moving, where that ball will land, etc.

So the conclusion from this, is that to some extent the information is compressed, removing noise and more.

The basis of machine learning is that human predictions and experiences are low dimensional. This is because we ourselves as humans can predict pretty well.

Mathematics : The mathematics of predictions are probability distributions $P(X)$ of low-dimensional support (manifold) in high-dimensional data space. Translated: we just look at a relevant subspace for the prediction.

Principle of parsimony : Learn a low-dimensional representation (from $10^4 - 10^{10}$ dimensions to $10^0 - 10^3$ dimensions)

8.1 How to reduce dimensionality

The problem is the high dimensionality of data

- Images: $\mathbb{R}^{3\times200\times300}$ (or more)
- Documents represented as bag-of-words. Each document counts occurances of each term. $tf(d,t)$. Represented by vector in $\mathbb{R}^{10000}$ or more.

The idea is to find the **most** important dimensions.

8.1.1 (Semi)Formal definition of dimensionality reduction

Given data set $\mathcal{D} = \{x_1, ..., x_N\} \subseteq \mathbb{R}^D$

Find a function $\hat{f} : \mathbb{R}^D \to \mathbb{R}^d$ with $d << D$ to derive a data set $\hat{\mathcal{D}} = \{\hat{f}(x_1), ..., \hat{f}(x_N)\} \subseteq \mathbb{R}^d$ such that the main characteristics are preserved

If the main characteristics are formalized, it's an optimization problem

$$\mathcal{L}(\hat{f}) = \sum_{i=1}^{N} (x_i - \hat{f}^{-1}(\hat{f}(x_i)))^2$$

This counts basically the 'information loss'. And we want to minimize that.

8.2 Dimensionality Reduction techniques

8.2.1 Linear Transformations Refresher

Given $x = (x_1, x_2)^T \in \mathbb{R}^2$

We can

- Scale by s_1, s_2 with
$$\begin{pmatrix} s_1 & 0 \\ 0 & s_2 \end{pmatrix} \cdot x$$
- Rotate by φ
$$\begin{pmatrix} \cos\varphi & -\sin\varphi \\ \sin\varphi & \cos\varphi \end{pmatrix} \cdot x$$
- Project onto one axis
$$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \cdot x = (x_1, 0)^T$$

8.2.2 Sample Covariance Matrix Refresher

Given data set $\mathcal{D} = \{x_1, ..., x_N\} \subseteq \mathbb{R}^D$

There are N data points, each with D dimensions. So we can represent everything as a $N \times D$ matrix.

$$X = \begin{pmatrix} x_{1,1} & \dots & x_{1,D} \\ \vdots & \ddots & \vdots \\ x_{N,1} & \dots & x_{N,D} \end{pmatrix}$$

And we can compute the **sample covariance matrix** $\hat{\Sigma} = [q_{j,k}] \in \mathbb{R}^{D \times D}$

$$q_{j,k} = \frac{1}{N-1} \sum_{i=1}^{N} (x_{i,j} - \overline{x}_j)(x_{i,k} - \overline{x}_k)$$

Intuitively, it measures how different features (dimensions) move together

- $(x_{i,j} - \overline{x}_j)$ asks, for observation i, is feature j above or below its average?
- $(x_{i,k} - \overline{x}_k)$ asks, for observation i, is feature k above or below its average?

Positive covariance means both features rise or decrease at the same time, like for example, as house size increases, it's price does also. Negative covariance is the inverse, car weight vs gas mileage. Zero (or close to it) means no relationship

8.2.3 Principal Component Analysis (PCA)

Assume that $\hat{f}$ is linear.

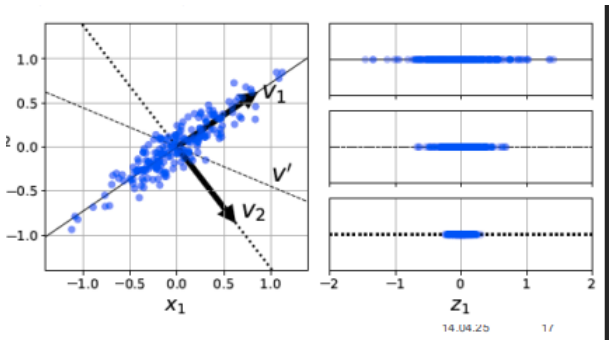
$$\hat{f}(x) = Vx$$

Then the loss is

$$\mathcal{L}(\hat{f}_V) = \mathcal{L}(V) = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{f}_V(x_i))^2$$

Idea of PCA: Given the usual dataset $\mathcal{D}$, first of all translate the data so that $\sum_{i=1}^{N} x_i = 0$
Represent it as the $N \times D$ matrix X

1. Rotate and project the data onto the **most important dimension**
2. Subtract this part
3. Repeat from 1



In this case, the most important dimension is V_1

Finding the best d axes (PCA) We are given data points:

$$x_1, x_2, \ldots, x_N \in \mathbb{R}^D$$

Goal:

- Find d orthonormal vectors (axes)

$$v_1, v_2, \ldots, v_d \in \mathbb{R}^D, \quad \|v_j\| = 1,\ v_i^\top v_j = 0\ (i \neq j)$$

- Find new coordinates for each data point:

$$z_i = (z_{i,1}, \ldots, z_{i,d}) \in \mathbb{R}^d$$

We approximate each point as:

$$x_i \approx \sum_{j=1}^{d} z_{i,j} v_j$$

The reconstruction error is:

$$\mathcal{L}(V, Z) = \frac{1}{N}\sum_{i=1}^{N}\left\| x_i - \sum_{j=1}^{d} z_{i,j} v_j \right\|^2$$

Goal: Minimize this error.

Finding the first axis We first solve for one direction v_1:

$$\mathcal{L}(v_1, z_{\cdot,1}) = \frac{1}{N}\sum_{i=1}^{N} \|x_i - z_{i,1}v_1\|^2$$

Expanding:

$$= \frac{1}{N}\sum_{i=1}^{N} \left(x_i^\top x_i - 2z_{i,1}v_1^\top x_i + z_{i,1}^2\right)$$

Optimize for $z_{i,1}$ (equal partial derivative to zero and clear up (computation skipped, directly give result)):

$$\frac{\partial}{\partial z_{i,1}}\mathcal{L}(v_1, z_{\cdot,1}) = 0 \quad \Rightarrow \quad z_{i,1} = v_1^\top x_i$$

(This is the projection of x_i onto v_1)

Plugging back:

$$\mathcal{L}(v_1) = \frac{1}{N}\sum_{i=1}^{N} \left(x_i^\top x_i - (v_1^\top x_i)^2\right)$$

$$= \underbrace{\text{const}}_{\text{Represents the } x_i^T x_i \text{ stuff}} - \frac{1}{N}\sum_{i=1}^{N} v_1^\top x_i x_i^\top v_1$$

Define empirical covariance:

$$\hat{\Sigma} = \frac{1}{N}\sum_{i=1}^{N} x_i x_i^\top$$

Then:

$$\mathcal{L}(v_1) = \text{const} - v_1^\top \hat{\Sigma} v_1$$

So minimizing $\mathcal{L}$ is equivalent to maximizing:

$$v_1^\top \hat{\Sigma} v_1 \quad \text{subject to } \|v_1\| = 1$$

Optimization via Lagrange multipliers

$$\mathcal{L}'(v_1) = v_1^\top \hat{\Sigma} v_1 - \lambda(v_1^\top v_1 - 1)$$

Set derivative to zero:

$$\hat{\Sigma} v_1 = \lambda v_1$$

Result:

- v_1 is an eigenvector of $\hat{\Sigma}$
- Choose the eigenvector with the **largest eigenvalue** (Why? See below)

We want to maximize $v_1^T \hat{\Sigma} v_1$ and we already saw that $\hat{\Sigma} v_1 = \lambda v_1$ so we can replace

$$v_1^T \hat{\Sigma} v_1 = v_1^T \lambda v_1 = \lambda \underbrace{v_1^T v_1}_{=1} = \lambda$$

So maximizing lambda maximizes the function from before.

Induction step (more axes) For additional axes:

- $v_2, v_3, \ldots, v_d$ must be orthogonal to previous ones
- Each maximizes variance in remaining directions

Result:

- v_j = eigenvector of $\hat{\Sigma}$
- Corresponding to the j-th largest eigenvalue

Final interpretation

- PCA finds directions of **maximum variance**
- Projects data onto a lower-dimensional space
- Minimizes reconstruction error

Intuition

- $d = D$: no information loss, just rotation
- $d < D$: compress data while keeping most important structure

9 Cluster Analysis

9.1 Goal of clustering

Identifying 'clusters' or 'groups' in the data, where objects in the same group are as similar as possible. Objects in different clusters should be as dissimilar as possible.

In **unsupervised learning**, no specific groups are given.

So the formal goal, is to find function $\hat{f}$, that classifies the objects into their clusters.

9.2 Clustering Task

Objects $X = \{x_1, ..., x_N\}$

Attributes for object $x_i = (x_{i,1}, ..., x_{i,m})^T$

Task: Find groups $\Omega = \{\omega_1, ..., \omega_k\}$ and find function $\hat{f} : X \to \Omega$

9.3 Cardinality/Number of clusters

Is either given explicitly, or to be found over that data.

9.4 Evaluation

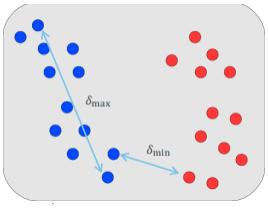
- Intrinsic: Evaluate quality of a cluster directly. Compactness, separation of groups...
- Extrinsic: Employs external knowledge. E.g partitioning cats and dogs, and then having a human checker

9.4.1 Intrinsic

Dunn Index : Notation of cluster separation

$$I_{Dunn}(\Omega) = \frac{\delta_{min}}{\delta_{max}}$$

- δ_{min} smallest inter-cluster distance
- δ_{max} largest intra-cluster distance



Higher is better. However, this biases towards one cluster per object.

Silhouette Coefficient : $s(i)$ for object x_i. It uses several components

- Average distance to all other objects in same cluster ω

$$a(i) = \sum_{x \in \omega, x \neq x_i} \frac{1}{|\omega| - 1} \delta(x_i, x)$$

- Average distance to some other cluster ω'

$$d(i, \omega') = \sum_{x \in \omega'} \frac{1}{|\omega'|} \delta(x_i, x)$$

- Average distance to closest other cluster

$$b(i) = \min_{\omega' \in \Omega, \omega' \neq \omega} d(i, \omega')$$

So the Silhouette coefficient is

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

with

$$-1 \leq s(i) \leq 1$$

A value close to one implies $b(i)$ is much larger than $a(i)$. Value closer to -1 represents the opposite.

The aggregate would be the **average silhouette coefficient** would be

$$\frac{1}{N} \sum s(i)$$

9.4.2 Extrinsic Metrics

We are given external ground truth of categories

$$C = \{c_i, ..., c_J\}$$

Idea: compare Ω and C

We define $n_j^{(i)}$:=number of objects from c_i clustered into ω_j

Purity : Ratio of strongest represented category

$$Purity(\omega_j) = \frac{1}{|\omega_j|} \cdot \max_{i=1,\dots,J} n_j^{(i)}$$

Aggregate over all clusters

$$Purity(\Omega) = \sum_{j=1}^{K} \frac{|\omega_j|}{N} Purity(\omega_j)$$

Example (categories c_1, c_2, c_3 are color coded green/yellow/red)

1. $\omega_1 = \{x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, x_{20}\}$
2. $\omega_2 = \{x_{12}, x_{13}, x_{14}, x_{15}, x_{16}, x_{17}, x_{18}, x_{19}\}$
3. $\omega_3 = \{x_{21}, x_{22}, x_{23}, x_{24}, x_{25}, x_{27}, x_{28}, x_{30}\}$
4. $\omega_4 = \{x_{26}, x_{29}\}$

Purity:

$$\text{Purity}(\omega_1) = \frac{10}{12} = 0.83$$

$$\text{Purity}(\Omega) = \frac{12}{30} \cdot 0.83 + \frac{8}{30} \cdot 1.0 + \frac{8}{30} \cdot 1.0 + \frac{2}{30} \cdot 1.0 = 0.93$$

Mutual Information : Mutual agreement between clusters and categories

$$MI(\Omega) = \frac{1}{N} \sum_{j=1}^{K} \sum_{i=1}^{J} n_j^{(i)} \cdot \log \frac{n_j^{(i)} \cdot N}{\sum_{m=1}^{J} n_j^{(m)} \sum_{l=1}^{K} n_j^{(i)}}$$

Rand Index For any pair of points (x_a, x_b), we distinguish:

- ss: same category, same cluster (agreement)
- dd: different category, different cluster (agreement)
- sd: same category, different cluster (disagreement)
- ds: different category, same cluster (disagreement)

$$I_{\text{Rand}}(\Omega) = \frac{ss + dd}{ss + dd + sd + ds}$$

Interpretation

- $I_{\text{Rand}} = 1$: perfect agreement
- $I_{\text{Rand}} = 0$: complete disagreement
- Measures fraction of correctly decided pairs

Example:

Example (categories c_1, c_2, c_3 are color coded green/yellow/red)

1. $\omega_1 = \{x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, x_{20}\}$
2. $\omega_2 = \{x_{12}, x_{13}, x_{14}, x_{15}, x_{16}, x_{17}, x_{18}, x_{19}\}$
3. $\omega_3 = \{x_{21}, x_{22}, x_{23}, x_{24}, x_{25}, x_{27}, x_{28}, x_{30}\}$
4. $\omega_4 = \{x_{26}, x_{29}\}$

Rand Index

- Agreements:
  - $ss = 103 = \binom{10}{2} + 1 + \binom{8}{2} + \binom{8}{2} + 1$
  - $dd = 280 = 10 \cdot 18 + 2 \cdot 10 + 8 \cdot 10$
- Disagreements
  - $sd = 32 = 2 \cdot 8 + 2 \cdot 8$
  - $ds = 20 = 2 \cdot 10$

$$I_{\text{Rand}}(\Omega) = \frac{\text{ss} + \text{dd}}{\text{ss} + \text{dd} + \text{sd} + \text{ds}} = \frac{383}{435} = 0.88$$

9.5 Loss Function vs Evaluation Metric

Don't confuse them! An evaluation metric is easy to understand, task specific truth that most humans find correct. A loss function is algorithm specific, complex and guides the algorithm to find the right solution.

9.6 K-Means

It's a general clustering algorithm.

Parameters

- K: Number of clusters
- Initial random seed

Given input data $\{x_i\}_{i=1}^N, x_i \in \mathbb{R}^d$ and $d, K \in \mathbb{N}$

Choose randomly K cluster centroid seeds $Z = z_i | z_i \in \mathbb{R}^d{}_{i=1}^K$
repeat{
for all objects x. assign x to cluster ω_i with minimal $\delta(x, z_i)$
For all clusters ω_i compute centroid $z_i = \frac{1}{|\omega_i|} \sum_{x \in \omega_i} x$
} repeat until centroids do not change

It's really easy, decently quick (O(n) for each iterations, usually done after 5-10 iterations).

However, susceptible to noise and outliers. Clusters always have convex form. k is sometimes hard to compute. Dependant on initial partition.

9.6.1 Evaluate Quality

Try different seeds and then evaluate quality for the best quality.

- Dunn Index
- Residual Sum of Squares

$$RSS(\Omega) = \sum_{j=1}^{K} \sum_{x \in \omega_j} \delta(x, z_j)^2$$

9.6.2 Variations

- (Random) Seed: Furthest Points/ Furthest modified points
- Stop Criterion: Small changes of the centroids. Fixed number of iterations
- K-Medioid: Non-standard metrics (string similarity)

9.6.3 Choice of K

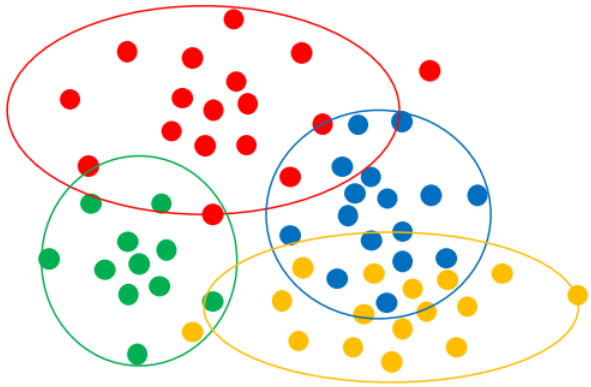
You have to have **some** knowledge about the data. More clusters always equates to less RSS, but it isn't always advantageous to have more clusters.

9.6.4 Problematic Configurations

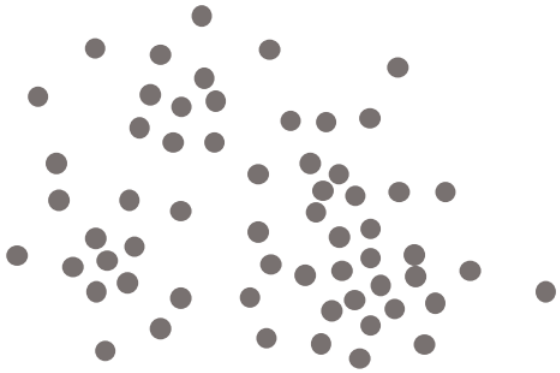
Problem	Solution
Outliers	Remove and treat separately
Empty Clusters	Split large cluster
Non-spheric shapes	Cannot be handled

10 Expectation Maximization

Idea: Assume that data is produced by a parameterized probability distribution.



The issue arises when we do not know the distributions and can only observe the raw data.



Expectation Maximization (EM) is a general clustering algorithm with a probabilistic approach and soft cluster assignment. It is a further generalization of K-Means.

Parameters:

- K: Number of clusters
- Initial random seed
- Model for the distribution

Refresher on probabilistic distributions: A probability density function (PDF) $f(x)$ satisfies:

$$\int_{\Omega} f(x)\,dx = 1$$

The cumulative distribution function (CDF) is defined as:

$$F(x) = P(X \leq x) = \int_{-\infty}^{x} f(t)\,dt$$

For multiple dimensions:

$$F(x_1, \ldots, x_n) = P(X_1 \leq x_1,\, \ldots,\, X_n \leq x_n) = \int_{-\infty}^{x_1} \cdots \int_{-\infty}^{x_n} f(t_1, \ldots, t_n)\,dt_n \cdots dt_1$$

10.1 Gaussian Models of Individual Distributions

10.1.1 Univariate Normal Distribution

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}}\,\exp\!\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

where μ is the mean and σ is the standard deviation.

10.1.2 Multivariate Normal Distribution

For a d-dimensional random vector $\mathbf{x}$, the density is:

$$f(\mathbf{x}) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}}\,\exp\!\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)$$

where $\boldsymbol{\mu}$ is the mean vector and Σ is the covariance matrix. The scalar standard deviation σ is simply replaced by Σ.

10.2 Model Estimation

Maximum Likelihood Estimation (MLE): Given a dataset $X = \{x_1, \ldots, x_n\}$, the **log-likelihood** is the logarithm of the probability of observing the data under parameters θ:

$$\log L(\theta \mid X) = \log\left(\prod_{x_i \in X} P(x_i \mid \theta)\right) = \sum_{x_i \in X} \log P(x_i \mid \theta)$$

We take the logarithm because it converts the product into a sum, which is numerically stable and easier to differentiate. Maximizing $\log L$ is equivalent to maximizing L since log is monotonically increasing.

Maximizing the log-likelihood yields the MLE estimators:

- **1-dimensional case:**

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \hat{\sigma}^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{\mu})^2$$

- ***d*-dimensional case:**

$$\hat{\boldsymbol{\mu}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i \qquad \hat{\Sigma} = \frac{1}{n}\sum_{i=1}^{n}(\mathbf{x}_i - \hat{\boldsymbol{\mu}})(\mathbf{x}_i - \hat{\boldsymbol{\mu}})^T$$

Note: The MLE for variance uses $\frac{1}{n}$ and is a *biased* estimator. The unbiased sample variance uses $\frac{1}{n-1}$ (Bessel's correction), but in the context of EM and MLE we use $\frac{1}{n}$.

The computation above assumes we know which object belongs to each cluster. Without cluster assignments, this is simply parameter estimation for a *single* Gaussian fitted to all the data — not a mixture. We need the EM algorithm to handle the unknown cluster memberships.

10.3 Cluster Assignment

Assumption — Independently and Identically Distributed (i.i.d.): A dataset is i.i.d. if every observation x_i is drawn independently from the same underlying probability distribution. *Independent* means the value of one observation does not affect any other; *identically distributed* means all observations follow the same distribution with the same parameters. This lets us write the joint likelihood as a simple product: $P(X \mid \theta) = \prod_i P(x_i \mid \theta)$.

For each object $x_i \in X$ we model **latent variables** z_{ij} indicating the probability that x_i belongs to cluster ω_j. The algorithm alternates two steps until convergence:

- **E-step (Expectation):** Given the current model parameters θ, compute the expected values of the latent variables z_{ij}. This yields the probability that object x_i belongs to cluster ω_j under the current model.
- **M-step (Maximization):** Given the current estimates of the latent variables z_{ij}, update the model parameters θ by maximizing the expected log-likelihood.

$$\mu_j = \frac{\sum_{x \in X} z_{ij}\, x_i}{\sum_{x \in X} z_{ij}}$$

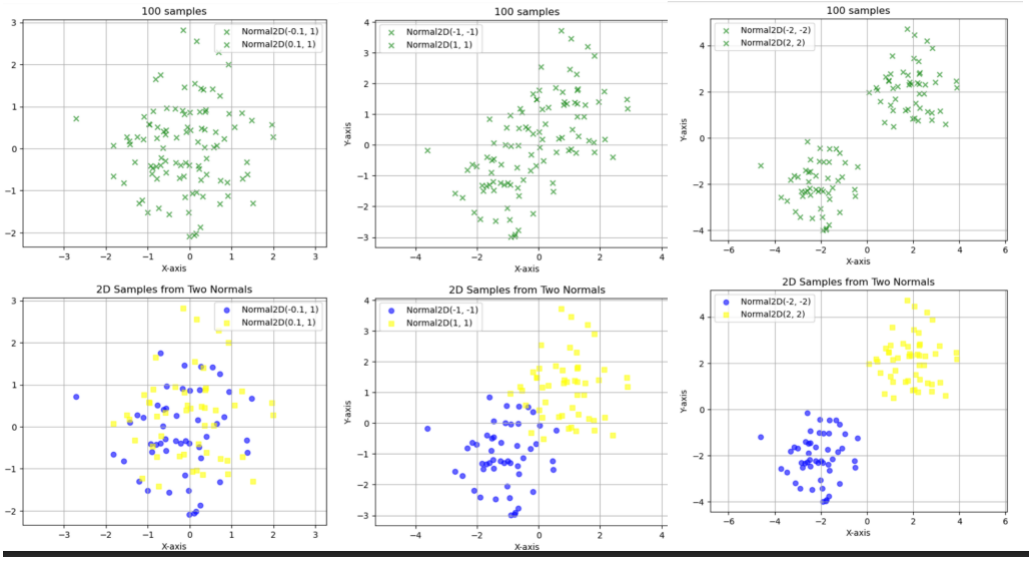
$$\sigma_j^2 = \frac{\sum_{x \in X} z_{ij}(x_i - \mu)^2}{\sum_{x \in X} z_{ij}}$$

10.3.1 Problem: Initialization

The E-step requires model parameters θ to estimate the latent variables, while the M-step requires latent variable estimates to update θ. This circular dependency means we must initialize θ before the first E-step, either randomly, via a heuristic (e.g. running K-Means first), or by another method.

10.3.2 Example

By varying the parameters of the Gaussian components we can observe how the cluster assignments change.



11 Linear Regression

11.1 Interpolation

Given a set of data points, find a polynomial that passes through all of them.

11.1.1 Lagrange Interpolation

$$\hat{f}(x) = \sum_{i=1}^{n} y_i \prod_{\substack{k=1 \\ k \neq i}}^{n} \frac{x - x_k}{x_i - x_k}$$

- Passes through every data point exactly
- Degree equals the number of data points minus one
- Very sensitive to outliers

11.1.2 Piecewise Linear Interpolation

Connect consecutive data points with straight lines.

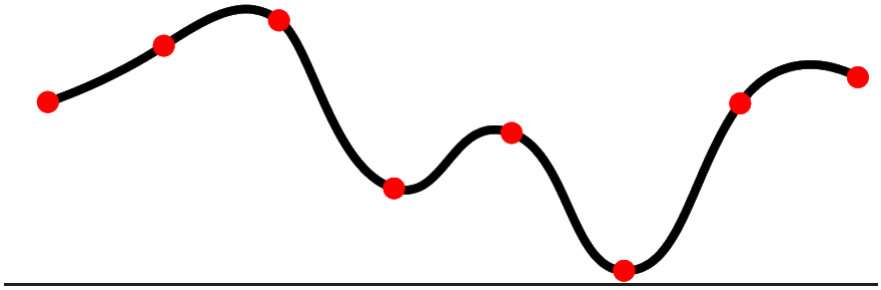
- Simple and fast
- Not smooth (sharp corners at data points)
- Sensitive to outliers

11.1.3 Spline Interpolation

Fit smooth polynomial pieces between data points, requiring the derivative to be continuous at each interior point x_i:

$$\forall\, i \in \{2, \ldots, n-1\}: \quad \lim_{\epsilon \to 0} \hat{f}'(x_i - \epsilon) = \lim_{\epsilon \to 0} \hat{f}'(x_i + \epsilon)$$

- Smooth transitions between segments
- Overly complex model $\Rightarrow$ tends to overfit outliers

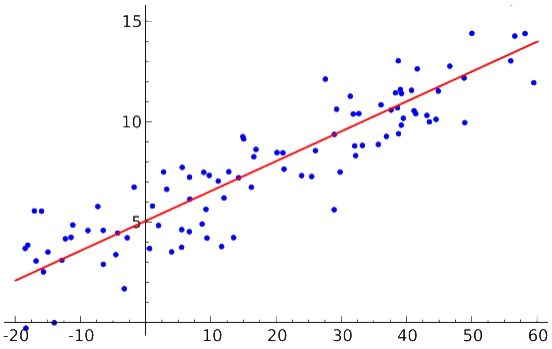


11.2 Regression

Unlike interpolation, regression does *not* require the function to hit every point. Instead it finds the best-fitting function while tolerating noise.

We assume data is generated by a linear function with some noise ϵ:

$$f(x) = \beta_1 x + \beta_0 + \epsilon$$



- Differences between predicted and actual values are called **residuals**
- Predictor variables (x) are also called **input variables**

11.2.1 Variants

- **Simple linear regression:** one predictor x_i, one response y_i
- **Multiple linear regression:** multiple predictors $\mathbf{x}_i = (x_{i,1}, \ldots, x_{i,d})$, one response y_i
- **Multivariate linear regression:** multiple predictors *and* multiple response variables

11.2.2 Multiple Linear Regression

Goal: estimate unknown parameters $\beta = (\beta_0, \ldots, \beta_d)^T$, assess their relevance, and estimate the error variance.

Scalar form:

$$y_i = \beta_0 + \beta_1 x_{i,1} + \cdots + \beta_d x_{i,d} + \epsilon_i$$

Vector form:

$$y_i = \bar{x}_i^T \beta + \epsilon_i$$

$$\begin{cases} \bar{x}_i = (1,\, x_{i,1}, \ldots, x_{i,d})^T \in \mathbb{R}^{d+1} \\ \beta = (\beta_0, \ldots, \beta_d)^T \in \mathbb{R}^{d+1} \\ \epsilon = (\epsilon_1, \ldots, \epsilon_n)^T \in \mathbb{R}^n \end{cases}$$

Matrix form:

$$\mathbf{y} = X\beta + \epsilon \qquad \text{where} \qquad X = \begin{pmatrix} \bar{x}_1^T \\ \vdots \\ \bar{x}_n^T \end{pmatrix} \in \mathbb{R}^{n \times (d+1)}$$

Optimization objective: Find $\hat{\beta}$ that minimizes the sum of squared residuals (**least squares**):

$$L^{ls}(\beta) = \|\epsilon(\beta)\|^2 = \|y - X\beta\|^2 = (y - X\beta)^T(y - X\beta)$$

Determine Extreme Value : We equalize the derivative to zero. This closed-form solution exists as long as X^TX is invertible

$$-2(y - X\beta)^T X = 0_d^T \Rightarrow 0_d^T = X^TX\beta - X^Ty \Rightarrow \hat{\beta}^{ls} = (X^TX)^{-1}X^Ty$$

This is the theoretical solution, but not commonly used because of inefficiency and computationally unstable. We use stochastic gradient descent to find where $X^TX\beta - X^Ty$ is zero in all its components

Beyond Linear Input Linear regression is called "linear" not because the input has to be linear, but because the model is linear in the **parameters** β. This means we are free to transform the raw inputs into something non-linear first, and then fit a regular linear model on top. Think of it as: instead of feeding the model plain x, you can square it, apply a Gaussian, etc. and *then* fit a line through the result.

We replace the inputs ($x_i \in \mathbb{R}^d$) with non-linear features $\phi(x_i) \in \mathbb{R}^k$, so that

$$y = \sum_{j=1}^{k} \phi_j(x)\,\beta_j + \epsilon \;=\; \phi(x)^T\beta + \epsilon$$

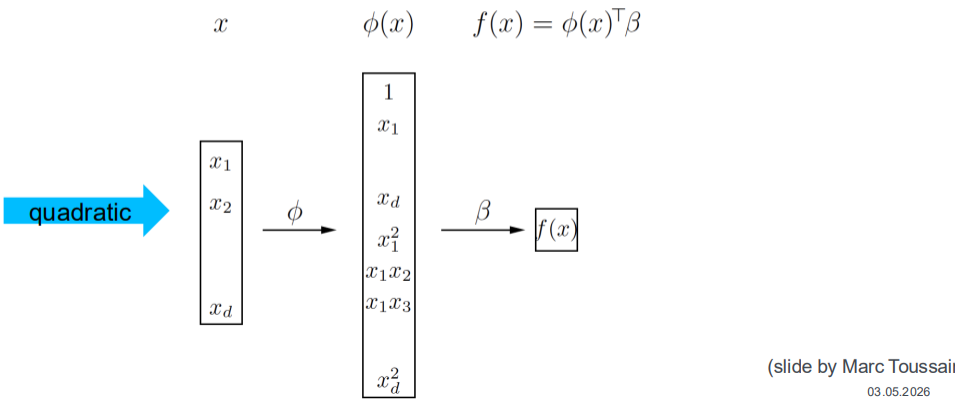
The catch: we still have to choose what ϕ looks like – that's the "guessing" part.

Example: Polynomial features The simplest non-linear trick. Instead of using just x, you add $x^2, x^3, \ldots$ as extra input columns. The model stays linear in β, but can now fit curved relationships.

Linear: $\phi(x) = (1, x_1, .., x_d) \in \mathbb{R}^{1+d}$

Quadratic: $\phi(x) = (1, x_1, .., x_d, x_1^2, x_1x_2, x_1x_3, .., x_d^2) \in \mathbb{R}^{1+d+\frac{d(d+1)}{2}}$

Cubic: $\phi(x) = (.., x_1^3, x_1^2x_2, x_1^2x_3, .., x_d^3) \in \mathbb{R}^{1+d+\frac{d(d+1)}{2}+\frac{d(d+1)(d+2)}{6}}$



(slide by Marc Toussai

03.05.2026

Example: Piece-wise features (in 1D) Instead of one global curve, we chop the x-axis into intervals ("pieces") and fit something simple inside each piece. We use the notation: [condition] = 1 if the condition is true, otherwise 0, and $[t]_+ = t$ if $t > 0$, otherwise 0(a ramp / ReLU).

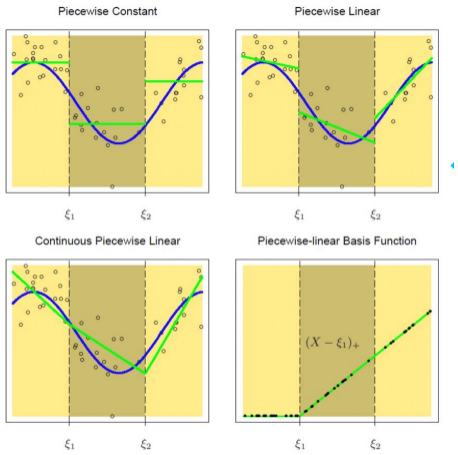
- **Piece-wise constant:** $\phi_j(x) = [\xi_j < x \leq \xi_{j+1}]$ flat step inside each bin, because for a single point x_i, its prediction would be

$$\sum_{j=1}^{N} \phi_j(x_i) \cdot w_j = w_j$$

  Because all the basis functions apart from i return zero.

- **Piece-wise linear:** $\phi_j(x) = (1,\, x)^T[\xi_j < x \leq \xi_{j+1}]$ a separate line in each bin (can jump at boundaries).

- **Continuous piece-wise linear:** $\phi_j(x) = [x - \xi_j]_+$ (with $\phi_0(x) = x$) lines that are forced to connect at each knot.



Example: Radial Basis Function (RBF) Pick a set of "center" points $\{c_j\}_{j=1}^{k}$ spread across your data. Each feature just measures *how close* x is to one center using a Gaussian bell-curve:

$$\phi_j(x) \;=\; b(x,\, c_j) \;=\; e^{-\frac{1}{2}\|x - c_j\|^2} \;\in [0, 1]$$

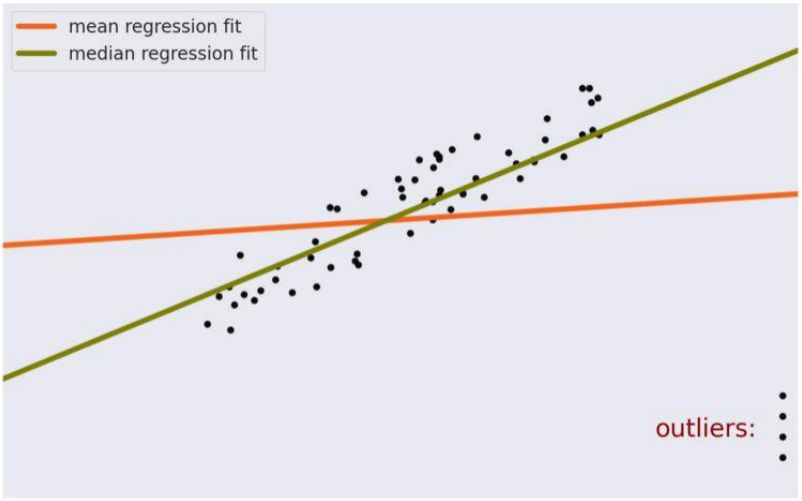
When x is right on top of c_j the feature is 1; the further away, the closer to 0. So the model ends up as a weighted sum of bumps.

11.3 Model Quality of Linear Regression

11.3.1 Mean vs Median

Mean regression minimises the squared-error loss $\ell = \epsilon^T \epsilon$.
Median regression minimises the absolute-error loss $\ell = \sum_i |\epsilon_i|$.

The squared loss punishes big errors *a lot* (because it squares them), so one extreme outlier can drag the whole fit away. The absolute loss treats every unit of error equally, making it far more robust to weird data points.



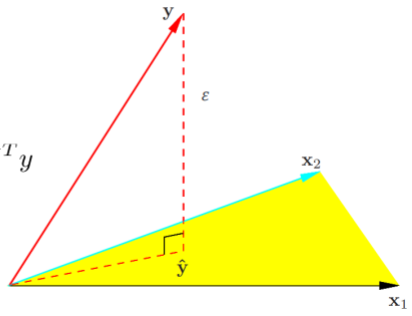
11.3.2 The Hat Matrix

The hat matrix $H = X(X^TX)^{-1}X^T$ is called that because it "puts a hat" on y:

$$\hat{y} = Hy$$

It is the projection matrix that maps the observed y onto the column space of X The issue is that $\hat{y}$ (our prediction) lives as a linear combination of the feature columns of X. This restricts $\hat{y}$ to a subspace. For that, we use this Hat matrix to project y to the subspace of $\hat{y}$ Each diagonal entry h_{ii} (called *leverage*) tells you how much observation i can pull the regression line towards itself – high leverage points are potentially dangerous.

$$\hat{y} = X\beta = X(X^TX)^{-1}X^Ty$$



11.3.3 Residuals

Residuals are what's left over after the model does its best: $e_i = y_i - \hat{y}_i$. If everything is fine they should look like *random noise* – no patterns.

Tukey–Anscombe plot: residuals e_i vs fitted values $\hat{y}_i$. Patterns in this plot signal problems:

- Residuals grow linearly with $\hat{y}$ – some systematic bias, wrong functional form.
- Residuals grow non-linearly with $\hat{y}$ – the true relationship is curved and the model misses it.
- Two distinct clouds with different spread – two sub-populations with different variances.
- A U-shape / arch – a quadratic term is missing from the model.

11.4 Investigating Model Assumptions

For the standard theory to hold, we assume:

1. Residuals are jointly normally distributed with $\mathbb{E}(\epsilon_i) = 0$ and $\text{Var}(\epsilon_i) = \sigma^2$ for all i. *(Errors are centred and not huge on average.)*
2. The variance σ^2 is the **same** for every observation (*homoscedasticity*). *(No observation is inherently noisier than another.)*
3. All x_i are observed **without noise**. *(Only y is noisy, not x.)*
4. Errors are uncorrelated: $\text{Cov}(\epsilon_i, \epsilon_j) = 0$ for $i \neq j$. *(One bad measurement doesn't infect the next.)*

Under these assumptions, since $y_i = x_i^T\beta + \epsilon_i$ and β, x are exact, every y_i has $\text{Var}(y_i) = \sigma^2$. If the residuals don't have mean zero, your model is missing something.

11.5 Goals of Linear Regression Analysis

1. **Good fit**: Small training error.

   Estimator Variance Given $\epsilon \sim \mathcal{N}(0, \sigma^2)$ we get $y \sim \mathcal{N}(X\beta,\, \sigma^2 I)$, and $\hat{\beta}$ is itself normally distributed:

   $$\hat{\beta} \sim \mathcal{N}\big(\beta,\, \sigma^2(X^TX)^{-1}\big)$$

   More data $\Rightarrow (X^TX)^{-1}$ shrinks $\Rightarrow \hat{\beta}$ wobbles less around the truth.

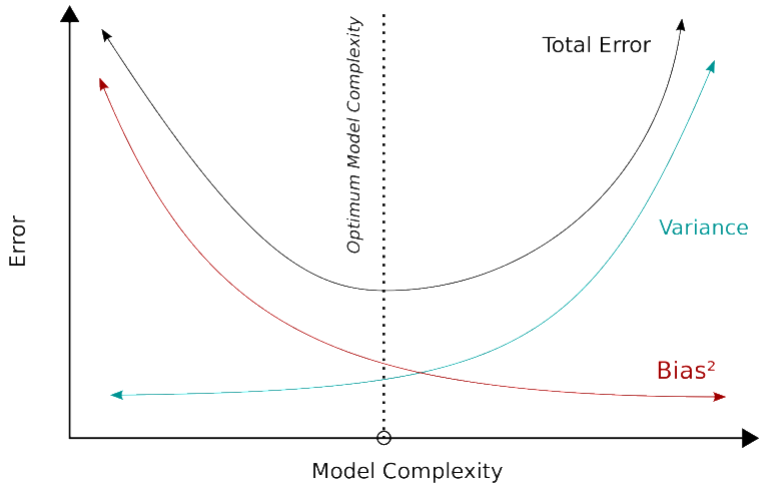
2. **Good prediction**: Small test error.

   Overfitting and generalisation A more expressive model always reduces training error, but it starts memorising the noise in the training data instead of the true signal – it then performs poorly on new data.

   Bias–Variance decomposition Let D be the training set, $\hat{f} = \hat{f}(x; D)$ our estimate, and f the true function. Using $\text{Var}[X] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$, $\mathbb{E}[\epsilon] = 0$, $\mathbb{E}[\epsilon^2] = \sigma^2$, and $y = f + \epsilon$, we can show:

   $$\mathbb{E}\Big[(y - \hat{f})^2\Big] \;=\; \underbrace{\sigma^2}_{\text{noise}} \;+\; \underbrace{\text{Var}[\hat{f}]}_{\text{variance}} \;+\; \underbrace{\text{Bias}[\hat{f}]^2}_{\text{bias}}$$

   Simple models: high bias (they miss the true shape), low variance (they don't change much with different training sets).
   Complex models: low bias, high variance (they overfit the noise). You want the sweet spot in the middle.



3. **Good parameter estimates**: What are the actual β_j values?

   With one predictor $y = \beta_0 + \beta_1 x$, standardise everything to zero mean, unit variance:

   $$x^* = \frac{x - \bar{x}}{\sigma_x}, \qquad y^* = \frac{y - \bar{y}}{\sigma_y}$$

   After standardisation $\beta_0^* = 0$, and the slope equals the **Pearson correlation**:

   $$\rho_{x,y} \;=\; \beta_1^* \;=\; \frac{1}{n}\sum_{i=1}^{n} x_i^* y_i^* \;=\; \beta_1 \frac{\sigma_x}{\sigma_y}$$

   So the standardised slope *is* the correlation – it tells you the strength and direction of the linear relationship, on a $[-1, 1]$ scale.

4. **Uncertainties and significance**: Are any predictors actually doing something useful?

   For each predictor j (corresponding to variable X_j) we run a hypothesis test:
   $H_{0,j}$: $\beta_j = 0$ (this predictor has no effect)
   $H_{A,j}$: $\beta_j \neq 0$ (it does have an effect)

   If we can reject $H_{0,j}$, predictor j is statistically significant.

11.6 Reducing Model Complexity

11.6.1 Subset Selection

Least-squares estimates have two practical problems:

1. **Prediction accuracy**: Low bias but potentially huge variance. Shrinking or zeroing some β_j can actually *improve* test error even though training error goes up slightly.

2. **Interpretation**: With dozens of predictors, the model is a mess to explain. Keeping only the strongest predictors gives a cleaner story.

Fewer variables = easier to interpret, often better generalisation.

11.6.2 Ridge Regression (L2)

Ridge adds a penalty on the size of β to prevent it from blowing up. Instead of minimising squared error freely, we force $\sum_{j=1}^{d} \beta_j^2 \leq t$ for some budget t:

$$\hat{\beta}^{\text{ridge}} = \arg\min_{\beta} \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{d} x_{i,j}\, \beta_j \right)^2 \quad \text{subject to} \quad \sum_{j=1}^{d} \beta_j^2 \leq t$$

Equivalently, via a Lagrange multiplier $\lambda \geq 0$:

$$\hat{\beta}^{\text{ridge}} = \arg\min_{\beta} \left[\sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{d} x_{i,j}\, \beta_j \right)^2 + \lambda \sum_{j=1}^{d} \beta_j^2 \right]$$

Note: $\arg\min_{\beta}$ refers to finding the β values which minimize the expression within the brackets. The first part is the Residual Sum of Squares, and the second part is our Penalty.

Larger $\lambda \Rightarrow$ stronger shrinkage $\Rightarrow$ simpler model. Setting $\lambda = 0$ recovers ordinary least squares.

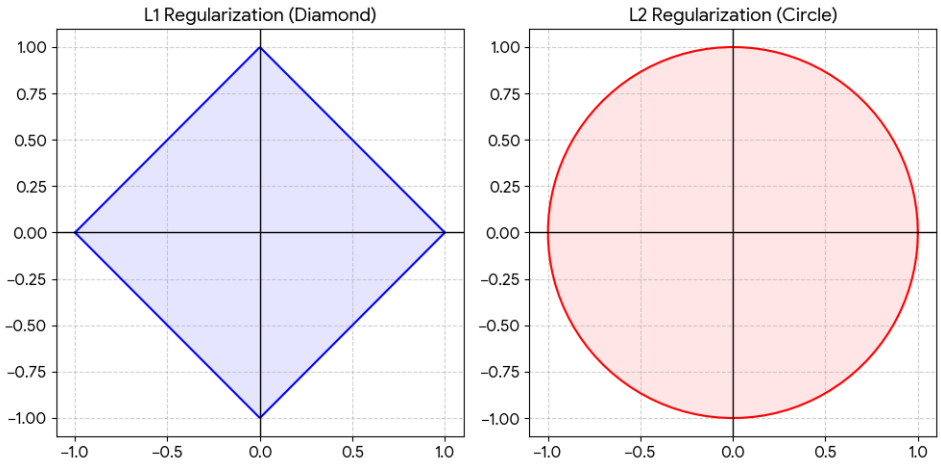
11.6.3 Lasso Regression (L1)

Instead of the Penalty being $\lambda \sum_{j=1}^{d} \beta_j^2$, it is $\lambda \sum_{j=1}^{d} |\beta_j|$

So the updated formula is

$$\hat{\beta}^{\text{lasso}} = \arg\min_{\beta} \left[\sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{d} x_{i,j}\, \beta_j \right)^2 + \lambda \sum_{j=1}^{d} |\beta_j| \right]$$

The Lasso has a diamond shape, so it is more biased for coefficients to be equal zero. It produces more sparcity. Ridge's coefficients almost never equal zero, it has a circle shape. Every feature stays in the model, but their influence is minimized



11.6.4 Choosing hyperparameter λ

So the lambda symbolizes how strong the penalty is. It controls the tradeoff between fitting the data well and keeping the model simple.

For $\lambda = 0$ we will always have a lower training data error, but we need to test the generalization error

k-fold cross validation



1: Partition data D in k equal sized subsets $D = \{D_1, .., D_k\}$
2: **for** $i = 1, .., k$ **do**
3: compute $\hat{\beta}_i$ on the training data $D \setminus D_i$ leaving out D_i
4: compute the error $\ell_i = L^{ls}(\hat{\beta}_i, D_i)/|D_i|$ on the validation data D_i
5: **end for**
6: report mean squared error $\hat{\ell} = 1/k \sum_i \ell_i$ and variance $1/(k\text{-}1)[(\sum_i \ell_i^2) - k\hat{\ell}^2]$

Choose the λ for which $\hat{l}$ is smallest.

12 Classification

12.1 Ground Truth Classifier

Given objects $x_i \in X$ described by m attributes from a set of domains $\mathcal{A} = \{A_1, \ldots, A_m\}$, such that:

$$\forall i : x_i = (x_{i,1}, \ldots, x_{i,m})^T$$

with $x_{i,k} \in A_k$.

We also define our set of target categories (classes) as $Y = \{l_1, \ldots, l_k\}$.

$$f : X \to Y$$

Our objective is to find the best possible approximation of f, denoted as $\hat{f}$. To achieve this, we restrict our search to a predefined hypothesis space (a family of functions) $\mathcal{F}$.

12.2 Defining the Task of Learning a Classifier

Given a dataset $\mathcal{D}$ containing n i.i.d. samples, $\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\} \sim X \times Y$, where each object x_i consists of m attributes and an observed category label $y_i \in Y$.

Given a parameterized function family $\mathcal{F}$, our goal is to find an optimal function $\hat{f}_\theta \in \mathcal{F}$ (where $\hat{f}_\theta : A_1 \times \cdots \times A_m \to Y$) that minimizes the empirical risk.

$$\hat{f}_\theta = \arg\min_{\tilde{f} \in F} \sum_{i=1,\ldots,n} l(\tilde{f}(x_i), y_i)$$

12.2.1 Empirical Risk Minimization (ERM)

While we ideally want to minimize the expected risk (true loss over the entire data distribution):

$$R(f) = \mathbb{E}_{(x,y)\sim P(X,Y)}[l(\hat{f}_\theta(x), y)]$$

we do not have access to the true joint probability distribution $P(X, Y)$. By the Law of Large Numbers, we can approximate the expected loss using the average loss over our training data $\mathcal{D}$. This core machine learning paradigm is known as **Empirical Risk Minimization**:

$$\arg\min_\theta \frac{1}{n} \sum_{i=1}^{n} l(\hat{f}_\theta(x_i), y_i) \overset{\text{LLN}}{\Rightarrow} \arg\min_\theta \mathbb{E}[l(\hat{f}_\theta(x), y)]$$

In summary, engineering a machine learning pipeline consists of four pillars:

1. **Model Design:** Define the hypothesis space $\hat{f}_\theta$.

2. **Data Pipeline:** Collect, clean, and augment data (x_i, y_i).
3. **Loss Function:** Design a suitable objective function l.
4. **Optimization Method:** Select an algorithm to solve $\arg\min_\theta$.

13 Validation and Evaluation

A strict separation between training and testing data is essential to assess generalization.

- **K-Fold Cross-Validation:** The training data can be split into folds for internal validation and hyperparameter tuning, but it must never contain or interact with the test data.
- **Test Data:** Reserved exclusively for the final evaluation of the model's generalization capabilities.

13.1 Combating Overfitting

To effectively tune and evaluate models, the dataset should be split into three non-overlapping subsets:

1. **Training Set:** Used to fit the model parameters (θ) by minimizing the loss
2. **Validation Set:** Used to evaluate performance during training and tune hyperparameters.
3. **Test Set:** Used strictly for the final performance evaluation.

13.2 Evaluation Metrics for Classifiers

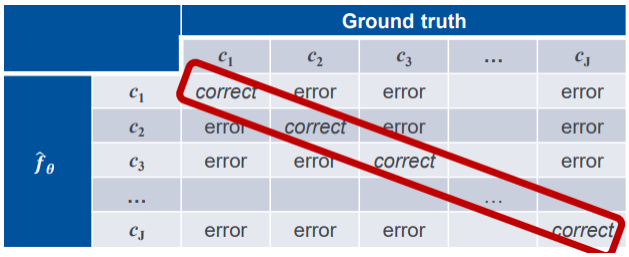
		Ground truth				
		c_1	c_2	c_3	...	c_J
$\hat{f}_\theta$	c_1	*correct*	error	error		error
	c_2	error	*correct*	error		error
	c_3	error	error	*correct*		error
	...				...	
	c_J	error	error	error		*correct*

Let a_{ij} denote the number of elements belonging to class c_j that were predicted as class c_i. We can define class-specific performance metrics:

1. **Recall** $r(c_i)$: The ratio of correctly predicted objects in class c_i out of all actual objects in c_i.
$$r(c_i) = \frac{a_{ii}}{\sum_{k=1}^{J} a_{ki}}$$
2. **Precision** $p(c_i)$: The ratio of correctly predicted objects in class c_i out of all objects predicted as c_i.
$$p(c_i) = \frac{a_{ii}}{\sum_{k=1}^{J} a_{ik}}$$
3. F_1**-Score** $F_1(c_i)$: The harmonic mean of precision and recall.
$$F_1 = \frac{2 \cdot r(c_i) \cdot p(c_i)}{r(c_i) + p(c_i)}$$

13.2.1 Global Metric: Accuracy

Accuracy measures the overall fraction of correct predictions across all J categories.



$$\text{Accuracy} = \frac{\sum_{i=1}^{J} a_{ii}}{\sum_{i=1}^{J} \sum_{j=1}^{J} a_{ij}}$$

13.2.2 Classification Errors

In binary classification, errors are categorized as follows:

1. **Type I Error (False Positive):** The model falsely predicts a positive condition for a negative instance.
2. **Type II Error (False Negative):** The model falsely predicts a negative condition for a positive instance.

13.3 Data Leakage

Data leakage occurs when information from outside the training dataset is inadvertently used to train the model. This causes the algorithm to create an overly optimistic performance baseline during validation, while failing to generalize to truly unseen data.

14 Naïve Bayes

Prerequisite Note: This method relies fundamentally on conditional probability and Bayes' Theorem. This is covered in the first slides regarding Bayes Classification

14.1 Bayes Classifier

The objective is to make classification decisions probabilistically based on an observation vector $o = (x_1, \ldots, x_m)$.

- **Soft Classification:** Outputs the complete posterior probability distribution over all possible classes.
$$\hat{f}_{\text{soft}}(o) = P(c \mid o)$$
- **Hard Classification:** Predicts the single specific class that yields the highest posterior probability.
$$\hat{f}_{\text{hard}}(o) = \arg\max_{c \in C} P(c \mid o)$$

By applying Bayes' Theorem, the posterior probability is calculated as:

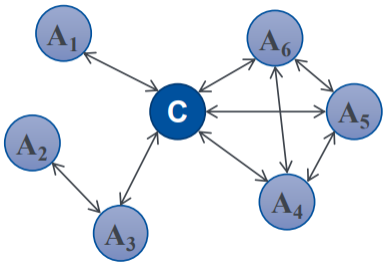
$$P(c \mid o) = \frac{P(c) \cdot P(o \mid c)}{P(o)} = \frac{P(c) \cdot P(x_1, \ldots, x_m \mid c)}{P(x_1, \ldots, x_m)}$$

Where:

1. $P(o)$: The **Evidence**. The marginal probability of the observation vector. It acts as a normalization constant and remains identical for all classes $c \in C$.
2. $P(c)$: The **Prior**. The baseline probability of a category before observing any data attributes.
3. $P(o \mid c)$: The **Likelihood**. The probability of observing features o given that the object belongs to class c.
4. $P(c \mid o)$: The **Posterior**. The updated probability that the observation belongs to class c after evaluating its attributes.

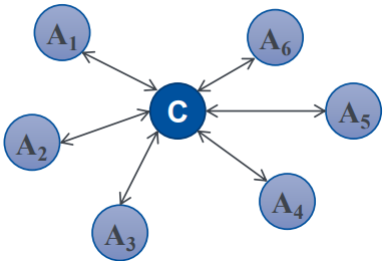
14.2 Full (Non-Naïve) Bayes Classifier

In a full Bayes framework, attributes and class labels are treated as mutually interdependent random variables. Modeling their full joint distribution requires tracking all complex combinations of dependencies.



14.3 Naïve Bayes Classifier

The **Naïve Bayes assumption** radically simplifies calculations by assuming that all attributes are conditionally independent of each other, given the class label c.



Using the chain rule of probability, the true likelihood expands exactly as:

$$P(o \mid c) = P(x_1, \ldots, x_m \mid c) = P(x_1 \mid x_2, \ldots, x_m, c) \cdot P(x_2 \mid x_3, \ldots, x_m, c) \ldots P(x_m \mid c)$$

Applying the conditional independence assumption simplifies this long expression to a clean product of independent marginal likelihoods:

$$P(x_1, \ldots, x_m \mid c) \approx \prod_{j=1}^{m} P(x_j \mid c)$$

Substituting this approximation back into Bayes' Theorem yields the standard formulation for the Naïve Bayes model:

$$P(c \mid x_1, \ldots, x_m) \approx \frac{P(c)}{P(x_1, \ldots, x_m)} \prod_{j=1}^{m} P(x_j \mid c)$$

14.3.1 Smoothing for Naïve Bayes Classifier

Laplace Smoothing : The issue is that with the formula above, as soon as one of the $P(x_j|c) = 0$, the entire product is zero.

$$P_{\text{Laplace}}(x_j = v \mid c_i) = \frac{|\{(x_1, \ldots, x_m, c) \in O : x_j = v \wedge c = c_i\}| + 1}{|\{(x_1, \ldots, x_m, c) \in O : c = c_i\}| + |V_j|}$$

What it represents: If a feature value never appears alongside class c_i in the training data, its probability is 0. Because Naïve Bayes multiplies all probabilities together, a single zero makes the entire global probability 0. We add +1 to the numerator (and scale the denominator by $|V_j|$, the set of values for attribute x_j) to guarantee every feature retains a baseline chance of occurring.

We can generalize this concept by substituting a custom parameter λ:

Generalized Additive Smoothing (Lidstone) :

$$P_{\text{Lidstone}}(x_j = v \mid c_i) = \frac{|\{(x_1, \ldots, x_m, c) \in O : x_j = v \wedge c = c_i\}| + \lambda}{|\{(x_1, \ldots, x_m, c) \in O : c = c_i\}| + |V_j| \cdot \lambda}$$

Implementation Detail (Log-Transformation): Calculating probabilities directly via:

$$P(c \mid x_1, \ldots, x_m) \propto P(c) \cdot \prod_{j=1}^{m} P(x_j \mid c)$$

is numerically unstable on computers. Multiplying many values close to zero causes floating-point *underflow* (rounding down to absolute 0). To stabilize this, we apply logarithms to turn multiplications into sums:

$$\log P(c \mid x_1, \ldots, x_m) \propto \log(P(c)) + \sum_{j=1}^{m} \log(P(x_j \mid c))$$

14.3.2 Handling Numerical (Continuous) Values

The Problem: For continuous fields (like precise height or weight measurements), the probability of observing an exact scalar value is zero ($P(\text{weight} = 67.0000\ldots) = 0$).

To fix this, we replace discrete counts with continuous distributions. Let X be a continuous random variable governed by a **probability density function** (PDF) $p(x) \in [0, \infty)$:

$$P(a \leq x \leq b) = \int_a^b p(x)\, dx \in [0, 1]$$

Its corresponding **cumulative probability distribution** (CDF) is defined as:

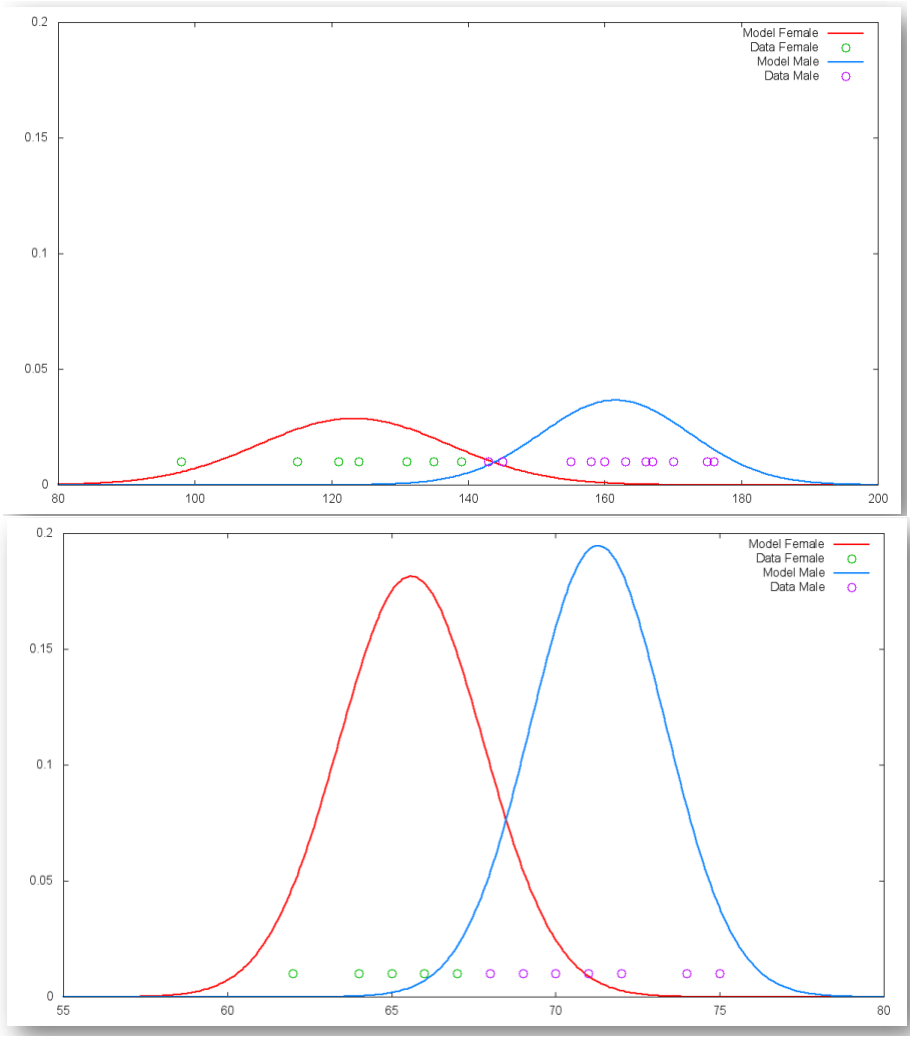
$$F(y) = P(x \leq y) = \int_{-\infty}^{y} p(x)\, dx \in [0, 1]$$

We assume continuous attributes follow a Gaussian (Normal) Distribution within each class:

$$\mathcal{N}(x \mid \mu, \sigma^2) = p(x \mid \mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

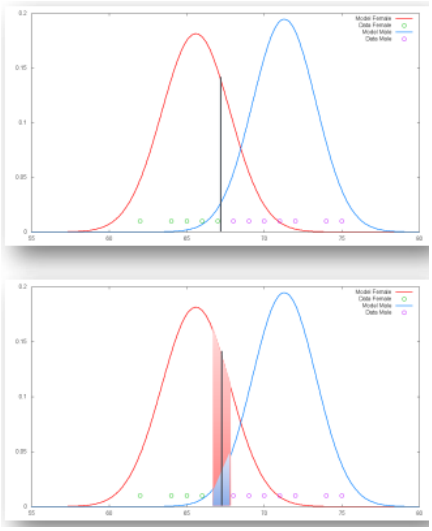
We determine the parameters using Maximum-Likelihood Estimation (MLE):

$$\mu_i = \frac{1}{n} \sum_o x_i, \quad \text{var}_i = \frac{1}{n-1} \sum_o (x_i - \mu_i)^2, \quad \sigma_i = \sqrt{\text{var}_i}$$



To utilize these continuous density functions for calculating class probabilities without getting 0, we evaluate a tiny interval around our target point x using a small margin ε:

$$P(\text{weight} = 67 \pm \varepsilon \mid c = f)$$



15 Logistic Regression

In Linear regression we predicted continous values, while here we predict discrete categories.

15.1 Bayes Error & Expected Prediction Error

> **What it represents:** The Expected Prediction Error (EPE) is a formal mathematical way of measuring the average penalty your model incurs for making incorrect predictions over a true data distribution.

If we utilize a **Squared Error Loss** function for numerical predictions, the error is formulated as:

$$EPE(\hat{f}) = E\left[(Y - \hat{f}(X))^2\right] = \int [y - \hat{f}(x)]^2 P(dx, dy)$$

Under this constraint, the absolute best mathematical prediction function matches the conditional mean:

$$\hat{f}(x) = E(Y \mid X = x)$$

If we switch to an **Absolute Error Loss** function:

$$EPE(\hat{f}) = E(|Y - \hat{f}(X)|) = \int |y - \hat{f}(x)| P(dx, dy)$$

The mathematically optimal prediction function matches the conditional median:

$$\hat{f}(x) = \text{median}(Y \mid X = x)$$

Note: Absolute loss is rarely used during training optimization because the absolute value brackets $|\dots|$ are not differentiable at zero.

15.1.1 Expected Prediction Error for Classification Task

Let a $K \times K$ penalty matrix L represent the structural loss $L_{G,\hat{G}}$ across a discrete set of categories $\mathcal{G}$. This matrix sets the precise penalty cost for classifying an object belonging to true category G into a predicted category $\hat{G}$:

$$L = \begin{pmatrix} 0 & L_{1,2} & \dots & L_{1,K} \\ L_{2,1} & 0 & \dots & L_{2,K} \\ \vdots & \dots & \ddots & \vdots \\ L_{K,1} & \dots & \dots & 0 \end{pmatrix}$$

Under a standard **Zero-One Loss** framework, any correct classification costs 0, while any incorrect classification incurs a static penalty of 1:

$$L = \begin{pmatrix} 0 & 1 & \dots & 1 \\ 1 & 0 & \dots & 1 \\ \vdots & \dots & \ddots & \vdots \\ 1 & \dots & \dots & 0 \end{pmatrix}$$

The Expected Prediction Error under a zero-one loss framework maps out to:

$$EPE(\hat{f}) = E\left[L_{G,\hat{G}(X)}\right] = E_X\left(\sum_{G \in \mathcal{G}} L_{G,\hat{G}(X)} P(G \mid X)\right)$$

To minimize this total error penalty, the optimal classification strategy resolves to selecting either:

$$\hat{G}(X) = \arg\min_{G \in \mathcal{G}} [1 - P(G \mid X = x)] \quad \iff \quad \hat{G}(X) = \arg\max_{G \in \mathcal{G}} P(G \mid X = x)$$

15.1.2 Optimal Bayes Classifier

Given a classification task over categories $\mathcal{G}$ where we assume full knowledge of the true underlying probability distribution $P(Y = y \mid X = x)$, the theoretical upper bound for performance is achieved by the **Optimal Bayes Classifier**:

$$\arg\max_y P(y \mid X = x)$$

15.2 Classification by Logistic Regression

Logistic regression learns a vector of continuous weights β to map input variables x_i directly onto a target prediction y. Unlike linear regression—where predictions can soar to infinity—logistic regression models a conditional **probability**, forcing the output y to look like a percentage strictly bound within the interval $[0, 1]$.

To achieve this constraint, the raw, unlimited linear score ($x^T\beta$) is routed through an S-shaped activation function called the **Logistic (Sigmoid) Function**:

$$\text{logistic}(z) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}}$$

By passing the linear dot product $x^T\beta$ through this squishing function, we force any raw positive or negative value safely into a valid probability format:

$$P(G = 1 \mid X = x) = \frac{e^{x^T\beta}}{1 + e^{x^T\beta}}$$

Evaluating this framework within a binary 2-Class scenario yields the following synchronized probability definitions:

- **Probability of Class 1:** $P(G = 1 \mid X = x) = \frac{e^{x^T\beta}}{1+e^{x^T\beta}}$
- **Probability of Class 2:** $P(G = 2 \mid X = x) = \frac{1}{1+e^{x^T\beta}}$

Because these two classes represent the entire available sample space, their sum equals exactly 1 (100%).

15.2.1 The Logit Transformation

To connect the bound probability interval $[0, 1]$ back up to an intuitive linear equation, we use the mathematical concept of **Odds** and **Logits**.

1. Probability vs. Odds: Probability measures the chance of an event happening divided by *all* possible outcomes. Odds measure the ratio of the event happening versus it *not* happening:

$$\text{Odds} = \frac{p}{1 - p} = \frac{\text{Probability of Event}}{\text{Probability of Non-Event}}$$

> **Intuition Check:** If a horse has an 80% (0.8) chance of winning a race, its probability is 0.8. However, its **odds** are $\frac{0.8}{1-0.8} = \frac{0.8}{0.2} = 4$. This means it is mathematically 4 times more likely to win than to lose (stated as '4 to 1 odds').

2. The Logit (Log-Odds): The **Logit** is simply the natural logarithm of those odds: $\log\left(\frac{p}{1-p}\right)$. If we feed our model's probability equations into this log-odds formula, the complex fractions unpack into a simple linear relationship:

$$\log\left(\frac{P(G = 1 \mid X = x)}{P(G = 2 \mid X = x)}\right) = \log\left(\frac{\frac{e^{x^T\beta}}{1+e^{x^T\beta}}}{\frac{1}{1+e^{x^T\beta}}}\right) = \log\left(e^{x^T\beta}\right) = x^T\beta$$

Why do this conversion?

- If an event has a perfect 50/50 chance ($p = 0.5$), the odds are $\frac{0.5}{0.5} = 1$. The logarithm $\log(1) = 0$. Score is exactly 0.
- If the probability drops below 50% ($p \to 0$), the odds drop, and the logit plunges toward $-\infty$. Score is negative.
- If the probability climbs above 50% ($p \to 1$), the odds spike, and the logit soars toward $+\infty$. Score is positive.

Thus, the 'Logit' is nothing more than the unlimited raw score calculated by our weights ($x^T\beta$). Logistic regression is just standard linear regression running silently behind the scenes to compute an unconstrained log-odds score, which is translated back into a practical probability at the very end.

15.2.2 Decision Boundary

Because a positive log-odds score maps to a probability $> 50\%$ and a negative score maps to a probability $< 50\%$, making a final classification decision becomes a straightforward threshold task:

- If $x^T\beta > 0 \longrightarrow P(G = 1 \mid X = x) > 0.5 \longrightarrow$ **Predict Class 1**
- If $x^T\beta < 0 \longrightarrow P(G = 1 \mid X = x) < 0.5 \longrightarrow$ **Predict Class 2**

The **Decision Boundary** is the exact geometric line (or hyper-plane) in space where the model's raw score balances at dead center zero ($x^T\beta = 0$).

Every point residing directly on this boundary line represents an ambiguous case where the model calculates a perfect 50/50 coin-flip probability. Stepping across this line in one direction flips the score positive (favoring Class 1); stepping across it the other way drops the score negative (favoring Class 2).

$$\{x \mid x^T\beta = 0\}$$

15.3 Logistic Regression: Multi-Class Case

We generalize our classification paradigm to a multi-class dataset $D = \{(x_i, y_i)\}_{i=1}^N$, where each feature vector belongs to $x_i \in \mathbb{R}^d$ and its corresponding categorical label falls within a set of K discrete classes: $y_i \in \{1, \ldots, K\}$.

Let $\mathbf{Y}$ denote an indicator assignment matrix. For each distinct target class y, we introduce a basis expansion or feature mapping $\phi(x)$ paired with a dedicated class-specific weight vector β_y:

$$f(x, y) = \phi(x)^T\beta_y$$

Which is the 'raw', unnormalized confidence of object x belonging to class y

Using these scoring terms, we define our conditional class probabilities via the multinomial Softmax mapping:

$$P(Y = y \mid X = x) = \frac{e^{f(x,y)}}{\sum_{y'} e^{f(x,y')}}$$

> **The Log-Odds Logit Equivalence:** This formulation preserves our linear framework. If we evaluate the log-ratio of probabilities between any two competing categories y and z, the global denominator terms completely cancel out, leaving a clean linear difference between their raw scores:
>
> $$f(x, y) - f(x, z) = \log\left(\frac{P(Y = y \mid X = x)}{P(Y = z \mid X = x)}\right)$$

15.3.1 Likelihood Formulation

Assuming our N dataset observations are drawn independently, the joint likelihood of a model parameterized by the global weight matrix β is computed as:

$$P(Y, X; \beta) = \prod_{i=1}^{N} P(y_i \mid x_i; \beta)$$

To convert these unstable products into computationally resilient additions, we take the natural logarithm to construct the mathematical **Log-Likelihood**:

$$\log P(Y, X; \beta) = \sum_{i=1}^{N} \log P(y_i \mid x_i; \beta)$$

—

15.4 Maximum Log-Likelihood Optimization

To optimize our model weights, our goal is to maximize this log-likelihood expression. In machine learning optimization routines, this is standardly framed as minimizing the regularized **Negative Log-Likelihood (NLL)** loss function:

$$L_{\text{logistic}}(\beta) = -\sum_{i=1}^{N} \log P(y_i \mid x_i) + \lambda \|\beta\|_2^2$$

Where $\lambda \|\beta\|_2^2$ represents an L_2 Tikhonov regularization penalty added to bound the weights and prevent overfitting.

15.4.1 Generalization via Cross-Entropy Loss

A highly generalized approach represents the categorical target variable using a **One-Hot Encoded Vector**. Let $\bar{\mathbf{y}}_i = \mathbf{e}_{y_i} = [0, \ldots, 0, 1, 0, \ldots, 0]^T$ denote this vector, where individual entry states are determined via an indicator function:

$$\bar{\mathbf{y}}_{i,z} = [y_i = z] = \begin{cases} 1 & \text{if } y_i = z \\ 0 & \text{otherwise} \end{cases}$$

Using this vector layout, we can rewrite the underlying Negative Log-Likelihood as a double summation over all classes:

$$L_{\text{nll}}(\beta) = -\sum_{i=1}^{N} \sum_{z=1}^{K} \bar{\mathbf{y}}_{i,z} \log P(Y = z \mid X = x_i)$$

This structure directly matches the mathematical definition of **Cross-Entropy** $H(p, q)$ evaluated between a true distribution p and an estimated model distribution q:

$$H(p, q) = -\sum_{z} p(z) \log q(z)$$

By mapping this relationship, we can express our global multi-class classification loss compactly as the total cross-entropy over our dataset:

$$L_{\text{nll}}(\beta) = \sum_{i=1}^{N} H\left(\bar{\mathbf{y}}_i, P(\cdot \mid X = x_i)\right)$$

Combining this with our L_2 structural regularizer yields the complete objective loss minimization framework:

$$L_{\text{logistic}}(\beta) = -\sum_{i=1}^{N}\sum_{z=1}^{K} \bar{\mathbf{y}}_{i,z} \log P(Y = z \mid X = x_i) + \lambda\|\beta\|_2^2 = \sum_{i=1}^{N} H\left(\bar{\mathbf{y}}_i, P(\cdot \mid X = x_i)\right) + \lambda\|\beta\|_2^2$$

—

15.5 Two-Class Log-Likelihood Optimization Analysis

To understand optimization mechanics deeply, we look at a binary setting ($K = 2$). Let our target labels be assigned such that if true category $Y = 1$, then $y = 1$, and if true category $Y = 2$, then $y = 0$. Let our prediction model be denoted by $\hat{f}(x_i; \beta)$.

Because of our assumption, $\hat{f}(x_i; \beta) = P(Y = 1|X = x_i; \beta)$, so $\log P(Y = y_i \mid X = x_i) = y_i \log(\hat{p}_i) + (1 - y_i)\log(1 - \hat{p}_i)$

The log-likelihood equation simplifies to:

$$\log P(Y, X; \beta) = \sum_{i=1}^{N} \Big[y_i \log \hat{f}(x_i; \beta) + (1 - y_i)\log(1 - \hat{f}(x_i; \beta)) \Big]$$

Plugging our specific logistic sigmoid mapping function ($\hat{f}(x_i; \beta) = \frac{e^{x_i^T \beta}}{1+e^{x_i^T \beta}}$) directly into this statement simplifies the log-likelihood down to:

$$\log P(Y, X; \beta) = \sum_{i=1}^{N} \Big[y_i x_i^T \beta - \log\left(1 + e^{x_i^T \beta}\right) \Big]$$

To solve for our optimal parameters, we calculate the first-order partial derivative (the Gradient vector) with respect to our weights β:

$$\frac{\partial \log P(Y, X; \beta)}{\partial \beta} = \sum_{i=1}^{N} \left[y_i x_i - \frac{e^{x_i^T \beta}}{1 + e^{x_i^T \beta}} \cdot x_i \right] = 0$$

And if we remember the definition of $\hat{f}$ and factor out x_i

$$\sum_{i=1}^{N} \Big(y_i - \hat{f}(x_i; \beta) \Big) x_i = 0$$

15.5.1 Numerical Optimization via the Newton-Raphson Method

Because this gradient equation contains non-linear transcendental components, setting it to zero **cannot be solved analytically** for β. Instead, the optimal parameter configurations must be discovered numerically using iterative optimization algorithms.

A highly robust approach is the second-order **Newton-Raphson Method**, which updates the weight parameters iteratively using the inverse Hessian matrix:

$$\beta^{\text{new}} = \beta^{\text{old}} - \left(\frac{\partial^2 \ell(\beta)}{\partial \beta \partial \beta^T} \right)^{-1} \frac{\partial \ell(\beta)}{\partial \beta}$$

Where $\ell(\beta) = \log P(Y, X; \beta)$ represents our target log-likelihood objective function.

16 Decision Trees

16.1 Measuring Distributions

16.1.1 Entropy

It is a measure of information content/uncertainty. Given random variable X we define entropy H(X) as

$$H(X) = -\sum_x P(X = x) \cdot \log_b P(X = x)$$

We usually assume b = 2

Alternative formulation

$$H(X) = E(I(X)) = E[-\log_b P(X)]$$

where $I(X)$ denotes information content of character x

Entropy of coin tosses : Assume fair coin toss

$$X = \{h, t\} \quad P(X = h) = P(X = t) = 0.5$$

What is the uncertainty of the coin toss? how much information have I received when I have been told the outcome?

$$H(X) = -0.5 \log_2 0.5 + 0.5 \log_2 0.5 = -0.5 \cdot (-1) + 0.5 \cdot (-1) = 1$$

1 bit

In the case of an unfair coin toss $P(X = h) = 1$

$$H(X) = -0 \log_2 0 + 1 \log_2 1 = 0$$

We learn no new information

Cross Entropy : For discrete probability distributions P, Q that are defined on the same probability space X. We define the **cross entropy**

$$H(P, Q) = E_P[-\log Q] = -\sum_{x \in X} P(X = x) \log Q(X = x)$$

How far is Q away from P? The issue with this is that $H(P, P) = H(P)$ which is usually non-zero, which intuitively doesn't make sense.

Relative entropy (Kullback-Leibler Divergence) : For discrete probability distributions P, Q that are defined on the same probability space X. We define the **relative entropy**

$$D_{KL}(P||Q) = \sum_{x \in X} P(x) \log \left(\frac{P(x)}{Q(x)} \right) = -\sum_{x \in X} P(x) \log \left(\frac{Q(x)}{P(x)} \right)$$

We can generalize this for continuous distributions with probability density functions p, q

$$D_{LK}(P||Q) = \int_{-\infty}^{\infty} p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$$

Interpretation: How far is the distribution Q from the distribution P?

Basically, $D(P||Q)$ measures the number of extra bits required to encode samples from P using a code optimized for Q.

$D(P||Q)$ is not symmetric, but $D(P||P) = 0$

Relation to maximum likelihood: Given data $\{(x_i, y_i)\}_{i=1}^N$ we maximize likelihood for our model over parameter configurations β

$$\prod_{i=1}^{N} P(y_i|x_i) = \prod_j q_j^{N_{y_j}}$$

Where

- N_{y_j}: total number of samples where $y = y_j$
- q_j: $P(y_j|x_j)$

This is not too clear for me, honestly.

each (x_i, y_i) in the training set has the same probability, therefore

$$\frac{1}{N} \log \prod_j q_j^{N_{y_j}} = \sum_j \frac{N_{y_j}}{N} \log q_j = -H(P(Y), P(Y|X))$$

Summary: **maximizing likelihood is equivalent to minimizing cross entropy**

16.2 Decision Trees

It is a tree with

1. Inner nodes
   - Label for attribute name
   - Outgoing edges with values from domain of attribute
2. Child Nodes of the inner nodes: Segment the data
3. Leaf Nodes: Assign one class label

The classification process is trivial, just follow the path that matches the observed attributes.

Can usually be nicely read by humans. To determine how good a tree is, we can use loss functions (not actually called that when talking about decision trees).

16.2.1 Inductive Construction of Decision Trees

The algorithm uses a **greedy approach**

- Tree grows from the root, the root is assigned no attributes (no **intension**) and all training objects (total **extension**).
- Inductive principle: Each node grows children that purifies the prediction quality of the training data.
  - Children increase the intension
  - Children partition the extension
- When stopping criteria is met, nodes are labeled with classes.

When constructing, the central metrics are **entropy** and **information gain**. How much uncertainty is in these children?

We apply entropy on the observed class labels, and reduce entropy in the child nodes. So when travelling down the tree, we want to reduce Entropy, therefore reducing the uncertainty. For this we want to compute the following

Expected entropy in the child nodes of node t: Computed as the probability of landing in the node times the entropy of the node

$$\sum_{n \in \text{children}} p(n) \cdot H(P_n)$$

and to compare it with the parent (and to see if it improved)

$$IG = H(P_t) - \sum_{n \in \text{children}} p(n) \cdot H(P_n)$$

With $p(n)$ is the ratio of children sent down, and P_n is the probability distribution of the class labels of the samples in child node n

Decision tree induction :

- Top down approach
  - Start from empty root node
  - apply **GrowTree** to root node
- GrowTree (for empty node)
  - If stop criterion matches node
    - Render node into label
    - Use most frequent class
  - Else
    - Render node into decision node
    - Choose attribute to maximize information gain
    - Introduce empty child nodes
    - Recursive call to child nodes
- Stop criteria: zero entropy, too few instances, or all attributes have equal values

Avoid overfitting : This algorithm favors attributes with many values. An example of this is that for example the attribute ObjectID has one value per object, so it perfectly recognizes the label. This is however unideal.

The solution for this is normalizing with respect to the distribution of attribute values.

$$H(x_i) = -\sum_v p(x_i = v) \cdot \log_2(P(x_i = v))$$

This is the entropy of an attribute, which increases if we split into a lot of buckets per possible value.

and we define the **gain ratio**(to be used instead of information gain to determine attribute)

$$IGR = \frac{IG(x_i)}{H(x_i)}$$

Alternative notation: For the normalized impurity decrease at a node t

$$\hat{\Delta} I(t) = \frac{\Delta I(t)}{E_d(t)} \quad \text{Where } E_d(t) \text{ is the entropy of distribution (intrinsic information)}$$

$$E_d(t) = -\sum_j P(t_j|t) \log_2 P(t_j|t)$$

$$P(t_j|t) = \frac{N_{t_j}}{N_t}$$

Overfitting : The algorithm for decision tree induction optimizes training error:

- Adding more nodes reduces training error
- Model becomes complex
- Risks overfitting

We want to make an idea of the error rate on new data.

generalization error

Estimating generalization error :

- For each node T with children $t \in T$ we determine the number of classification errors on raining data $(e(T), e(t))$ and the number of objects assigned to this node $(n(T), n(t))$
- Overall error rate is defined as
$$\frac{1}{n(T)} \sum_{t \in T} e(t)$$
- Add a penalty for each node (pessimistic error estimate)
$$\frac{1}{n(T)} \sum_{t \in T} e(t) + \lambda$$

Tree pruning :

- During construction we extend the stop criteria, so that we require minimum information gain, and require minimum number of instances to be covered by a node
- During post processing, remove leaf nodes and optimize other criteria (estimates for generalization error). Also prune excessively deep sub-trees
- During selection, construct different trees (using e.g subsets of training data). We select the best and smallest tree.

16.2.2 Attribute Splitting

So far we had one child node per value. Some attribute types are for example continous or have tons of values.

So for ordinal values, we can create meaningful splits of the data.

For numerical data, we use inbetween values for the split. We can also use bins to turn numerical data into ordinal data. When binning, we can do equi-width or equi-depth bins. Either same range per bin or same number of values per bin.

16.2.3 Pros and Cons of decision trees

Pros

1. Interpretable
2. Non-parametrics
3. Low complexity for prediction
4. Invariant to feature scaling
5. Can handle missing data

Cons

1. Splits are aligned with respect to axes

17 Bagging and Boosting

For an input x_0, assuming a target $y_0 = f(x_0) + \epsilon$ (where $\mathbb{E}[\epsilon] = 0$ and $\text{Var}(\epsilon) = \sigma^2$), the expected prediction error under squared error loss is decomposed as:

$$\text{Err}(x_0) = \mathbb{E}\left[\left(y_0 - \hat{f}(x_0)\right)^2\right] = \sigma^2 + \text{Bias}^2\left(\hat{f}(x_0)\right) + \text{Var}\left(\hat{f}(x_0)\right)$$

where σ^2 is the **Irreducible Error**.

When a model overfits (e.g., fully grown decision trees), it exhibits low bias but high variance. Ensemble methods combine multiple base models to optimize this tradeoff.

1. **Bagging (Bootstrap Aggregating)**
   - **Mechanism:** Generates B parallel datasets via bootstrap sampling (sampling with replacement from the training data). Trains an independent base estimator $\hat{f}^b$ on each sample.
   - **Objective:** Reduces **variance** by averaging uncorrelated predictors without changing the bias.
   - **Regression:** Computes the arithmetic mean of all predictions: $\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^b(x)$.
   - **Classification:** Aggregates predictions via majority voting.
2. **Boosting**
   - **Mechanism:** Trains base estimators sequentially. In each iteration, the algorithm re-weights the training data to assign higher importance to misclassified or high-residual samples from previous steps.
   - **Objective:** Primarily reduces **bias** by sequentially converting weak learners into a strong ensemble.
   - **Aggregation:** Computes a weighted combination of all sequential predictors based on their training accuracy: $\hat{f}_{\text{boost}}(x) = \sum_{m=1}^{M}\alpha_m f_m(x)$.

17.1 Constructing a Random Forest

A random forest is constructed by multiple decision trees and the final decision is based on the **majority vote** of all the trees.

17.1.1 Reducing Variance

Averaging B i.i.d. random variables each with mean μ and variance σ^2 yields:

- Mean μ — unchanged bias.
- Variance $\frac{1}{B}\sigma^2$ — lowered variance.

If the random variables are instead pairwise correlated with correlation coefficient ρ, the aggregated variance becomes:

$$\left(\rho + \frac{1-\rho}{B}\right)\sigma^2$$

17.1.2 Bagging Mechanism in Random Forests

If the variance of a prediction function is too high, we can use **random forests** as a special method for bagging uncorrelated trees.

```
Define number of trees B

for b = 1 to B:
  Draw a bootstrap sample Z* of size N from the original dataset of size N
  (Bootstrap sampling: Pick a random value, record it, and put it back)

  Grow a random-forest tree T_b using the Z* sample.
  For each node, repeat until minimal node size n_min is reached:
    Select l* (where l* < √l) attributes at random from all l attributes
    Using Gain Ratio, split the node into children nodes

Output set of trees T_b
```

Using Random Forest for Classification: Let $\hat{f}_b(x)$ be the class prediction of x by the b-th tree:

$$\hat{f}(x) = \arg\max_i \sum_{b=1}^{B} \left[\hat{f}_b(x) = i\right]$$

Generalization Error of an Ensemble of Decision Trees:

- The bias of the ensemble is identical to the bias of a single randomized tree, but typically higher than the bias of a non-randomized tree.
- Stronger randomization: $\text{Var}(\hat{f}) \to 0$ as the pairwise correlation $\rho \to 0$.
- Weaker randomization: $\text{Var}(\hat{f}) \to$ the variance of a single non-randomized tree.

Thus, randomization increases individual tree bias but significantly decreases the variance of the ensemble. The goal is finding the optimal trade-off.

Advantages:

1. Better accuracy than a single decision tree.
2. Robust to outliers and missing data.
3. Robust to irrelevant attributes due to random feature selection.
4. Distribution-free (no scaling needed).
5. Reduces overfitting.
6. Highly accurate for large datasets.

The primary disadvantage is that model interpretability decreases.

17.1.3 Interpreting Random Forests

Attribute Importance Score: Mean Decrease of Accuracy (MDA) Consider the out-of-bag (OOB) samples (samples not included in the bootstrap training set for a given tree). For each tree, we permute the values of the target attribute with random noise. We evaluate how much a metric like accuracy drops after this permutation. Computing the mean decrease of accuracy across all trees reveals attribute importance; a high MDA indicates a highly important feature.

Attribute Importance Score: Mean Decrease of Impurity (MDI) For a variable X_j, we define its importance with respect to an ensemble of B trees φ_m as:

$$\text{Imp}(X_j) = \frac{1}{B}\sum_{m=1}^{B}\sum_{t\in\varphi_m}\mathbb{I}(j_t = j)\,[p(t)\Delta i(t)]$$

Equation Breakdown:

- $\mathbb{I}(j_t = j)$: An indicator function that outputs 1 if node t splits on feature j, and 0 otherwise.
- $p(t) = \frac{N_t}{N}$: The proportion of total samples passing through node t.
- $\Delta i(t)$: The reduction in impurity achieved by the split at node t, calculated as:

$$\Delta i(t) = i(t) - \frac{N_{t_L}}{N_t}i(t_L) - \frac{N_{t_R}}{N_t}i(t_R)$$

  $i(t)$ can be some measure of impurity, either entropy or some other measure

Intuition: A feature is important if it is chosen frequently for splits, occurs at nodes containing a high volume of samples, and yields a large drop in impurity (disorder).

Compared to MDA, MDI is used more frequently because it is significantly faster and computationally easier to calculate, usually correlating closely with MDA.

17.1.4 Computational complexity

	Training	Prediction
Decision Tree	$O(l \cdot N \log^2(N))$	$O(l)$
Random Forest	$O(B \cdot \hat{l} \cdot \hat{N} \log^2(\hat{N}))$	$O(B \cdot \hat{l})$
Extra Tree	$O(B \cdot \hat{l} \cdot \hat{N} \log(\hat{N}))$	$O(B \cdot \hat{l})$

- B: Number of trees
- $\hat{l}$ number of variables randomly drawn at each node
- $\hat{N} = 0.632N$

18 Explainable Machine Learning

18.1 Local Feature Attributions by Shapley Values

The core concept of Shapley values is to evaluate a feature's importance by combining it into various subsets (coalitions) of other features to observe its marginal contribution to the model's prediction output.

$$s_j(\text{val}) = \sum_{T\subseteq N\setminus\{j\}} \frac{|T|!\,(p-|T|-1)!}{p!}\left(\text{val}(T\cup\{j\}) - \text{val}(T)\right)$$

Where:

- $N = \{1,\ldots,p\}$ is the complete set of all p features, and x is the specific instance vector to be explained.
- $\text{val}_{f,x}(T)$ represents the prediction for the feature values in subset T, marginalized over all features not included in T:

$$\text{val} : 2^N \to \mathbb{R}, \quad \text{val}_{f,x}(T) = \mathbb{E}_{X|X_T=x_T}[f(X)] - \mathbb{E}_X[f(X)]$$

  Intuition: This represents the conditional expected prediction given the features in subset T, minus the global baseline average prediction of the model across the entire dataset.

18.1.1 Properties of Shapley Values

1. **Efficiency Property:** The sum of all feature attributions equals the difference between the specific local prediction $f(x^*)$ and the global expected model value:

$$\sum_{j \in N} s_j(f, x^*) = f(x^*) - \mathbb{E}[f(X)]$$

2. **Uninformativeness (Null Player) Property:** A feature that never alters the predicted value regardless of the coalition it joins receives a Shapley value of zero:

$$\forall T \subseteq N \setminus \{j\},\ f(T \cup \{j\}) = f(T) \implies s_j(f, x) = 0$$

19 Decision Stumps

A decision stump is a single-level decision tree with two branches. Given training data $\{(x_i, y_i)\}_{i=1}^N$ with $x_i \in \mathbb{R}^d$ and $y_i \in \{-1, 1\}$, a decision stump $\phi_{j,s}$ is parameterized by:

- Polarity $p \in \{-1, 1\}$
- Threshold $s \in \mathbb{R}$
- Feature index $j \in \{1, \ldots, d\}$

$$\phi_{j,s}(x) = p \cdot \text{sign}(x_j - s) = \begin{cases} p & \text{if } x_j \geq s \\ -p & \text{else} \end{cases}$$

19.1 Selecting the Best Decision Stump

The training error of a decision stump is defined as:

$$\hat{\text{Err}}(\phi_{j,s}) = \frac{1}{N} \sum_{i=1}^{N} [\phi_{j,s}(x_i) \neq y_i] = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}\left(y_i \cdot p(x_{i,j} - s) \leq 0\right)$$

We optimize the parameter set by finding:

$$\arg \min_{j \in \{1,\ldots,d\},\ s \in \{x_{i,j} | i \in \{1,\ldots,N\}\}} \hat{\text{Err}}(\phi_{j,s})$$

Intuitively, a decision stump selector acts as a base **weak learner** that performs only slightly better than a random guess.

20 AdaBoost

20.1 AdaBoost Scheme

Given a collection of base feature functions $\phi_j : \mathbb{R}^d \to \{-1, 1\}$ and an expansion weight vector $\theta = [\theta_1, \theta_2, \ldots]^T$, the ensemble classifier is formulated as:

$$\hat{f}_\theta(x) = \text{sign}\left(\sum_{j=1}^{\infty} \theta_j \phi_j(x)\right)$$

Intuitively, we want to classify x to one of two buckets $(-1, +1)$. We have a collection of the feature functions ϕ_j, which are decent guesses, but don't do much. We also have the weight vector θ that for each base function we get a weight or trust score. The total output is just the sign of the sum of all weighted guesses.

20.2 Discrete AdaBoost Algorithm

1. Initialize training sample weights $w_i = \frac{1}{N}$ for $i = 1, \ldots, N$. (Each data popint is worth the same)
2. **Repeat until convergence of** $\theta^T \phi(x)$:
   - Fit a weak classifier $\phi_m(x)$ to the training data minimizing the weighted error using weights w_i.
   - Compute the normalized weighted error:

     $$\hat{\text{Err}}(\phi_m) = \frac{\sum_{i=1}^{N} w_i \left[\phi_m(x_i) \neq y_i\right]}{\sum_{i=1}^{N} w_i}$$

     Measures the fraction of the total weight that was misclassified by ϕ_m
   - Compute the classifier stage weight via log-odds: $\theta_m = \frac{1}{2} \log \frac{1-\hat{\text{Err}}(\phi_m)}{\hat{\text{Err}}(\phi_m)}$.

     This is something like a trust score, low error gives us high trust score. High error (> 0.5) gives a negative value, basically flipping the models prediction.
   - Update instance weights: $w_i := w_i \cdot e^{\theta_m [y_i \neq \phi_m(x_i)]}$ for $i = 1, \ldots, N$.

     This increases the weight of misclassified data points.

     So if we were mostly accurate, we make a great effort to fix the remaining misclassifications. If it was weak in it's accuracy, we don't make too much effort to fix the misclassifications.
3. Output the final ensemble model: $\hat{f}(x) = \text{sign}\left[\sum_{m=1}^{M} \theta_m \phi_m(x)\right]$.

This process amplifies the weights of misclassified samples and scales down correctly classified ones in each successive step.

20.3 Understanding AdaBoost via Forward Stagewise Additive Modeling

Consider a general additive expansion model using base functions $b(x; \beta_m)$:

$$\hat{f}(x) = \sum_{m=1}^{M} \theta_m b(x; \beta_m)$$

Where θ_m are stage coefficients and β_m are base model parameters. Global optimization of all parameters simultaneously is too computationally expensive. Instead, **Forward Stagewise Additive Modeling (FSAM)** fits one basis function at a time.

20.3.1 Forward Stagewise Additive Modeling Algorithm

1. Initialize the base model state: $\hat{f}_0(x) = 0$.
2. **Repeat until convergence**:
   - Compute the optimal stage parameters for the next isolated addition:

     $$(\theta_m, \beta_m) = \arg\min_{\tilde{\theta}, \tilde{\beta}} \sum_{i=1}^{N} L\left(y_i, \hat{f}_{m-1}(x_i) + \tilde{\theta} b(x_i; \tilde{\beta})\right)$$

     Finding the best **new** parameters to minimize the loss with what we already have
   - Incrementally update the ensemble model:

     $$\hat{f}_m(x) = \hat{f}_{m-1}(x) + \theta_m b(x; \beta_m)$$

Connection to AdaBoost: AdaBoost is a specific implementation of Forward Stagewise Additive Modeling. It optimizes parameters under an **Exponential Loss Function**:

$$L\left(y, \hat{f}(x)\right) = e^{-y\hat{f}(x)}$$

At step m, the optimization objective resolves to:

$$(\theta_m, \phi_m) = \arg\min_{\tilde{\theta},\tilde{\phi}} \sum_{i=1}^{N} e^{-y_i\left(\hat{f}_{m-1}(x_i)+\tilde{\theta}\tilde{\phi}(x_i)\right)} = \arg\min_{\tilde{\theta},\tilde{\phi}} \sum_{i=1}^{N} w_i^{(m)} e^{-y_i\tilde{\theta}\tilde{\phi}(x_i)}$$

Where $w_i^{(m)} = e^{-y_i\hat{f}_{m-1}(x_i)}$ corresponds to the step-by-step updated instance weights generated by AdaBoost.

And for any $\tilde{\theta} \geq 0$ this is equivalent.

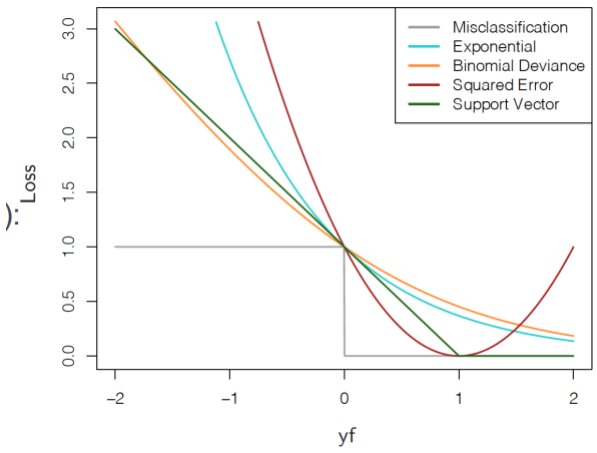
$$\arg\min_{\tilde{\theta},\tilde{\phi}} \sum_{i=1}^{N} w_i^{(m)}[y_i \neq \tilde{\phi}(x_i)]$$

21 Understanding losses

21.1 Comparing Loss functions for binary classification

Note: As I understand it, this binary classification has outputs in the form $y_i \in \{-1, 1\}$ and the classifier

$$\hat{f}(x_i) = \begin{cases} > 0 & \text{if } y_i = 1 \\ < 0 & \text{if } y_i = -1 \end{cases}$$



- Misclassification

$$[\text{sign}(\hat{f}) \neq y]$$

- Exponential

$$e^{-y\hat{f}}$$

  Punishes wrong predictions exponentially, so problematic when bayes error is high

- Cross entropy (Binomial deviance)

$$\log(1 + e^{-2y\hat{f}})$$

  Deals better with high Bayes error in the dataset

- Squared error

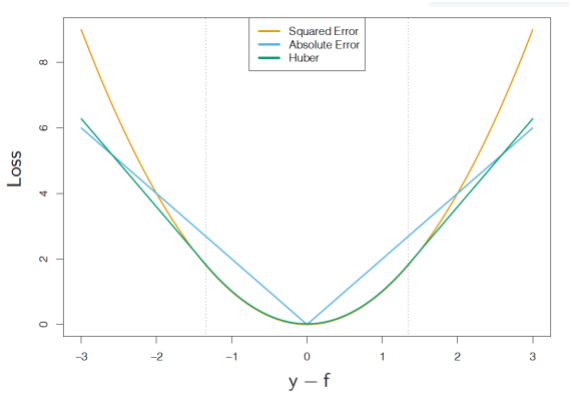
$$(y - \hat{f})^2$$

  It does not decrease continuously, so unnecessary attention on correctly classified entities.

- Hinge loss (Support vector)

$$\max(0, 1 - y\hat{f})$$

21.2 Comparing loss functions for regression



- Squared error

$$(y - \hat{f})^2$$

  Less robust against outliers

- Absolute error

$$|y - \hat{f}|$$

- Huber

$$L\left(y, \hat{f}(x)\right) = \begin{cases} (y - \hat{f}(x))^2, & |y - \hat{f}(x)| < \delta \\ 2\delta|y - \hat{f}(x)| - \delta^2 & \text{otherwise} \end{cases}$$

21.3 Takeaway

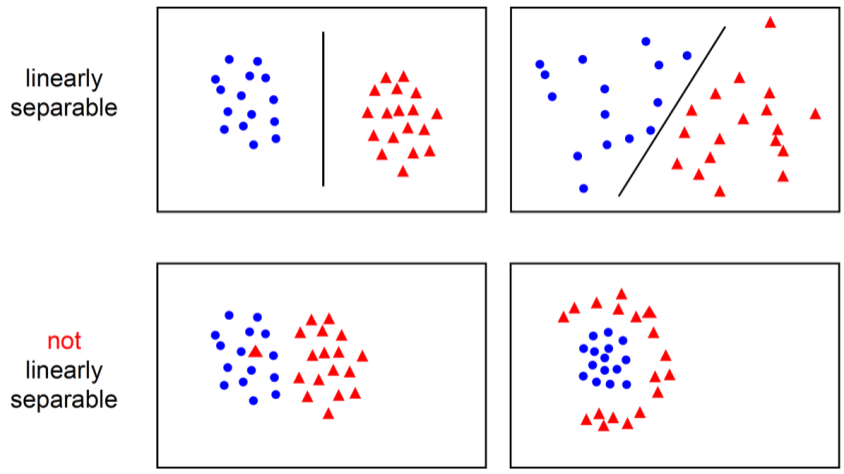
Some loss functions perform badly in certain situations

22 Support Vector Machines

22.1 Perceptron Algorithm

22.1.1 Linear Separability

As the name says, if you can partition the data with a 'line'



A linear classifier has the form

$$\hat{f}(x) = w^T x + b$$

(basically the point-slope function but in multiple dimensions). w is the normal to the line (weight vector), and b is the bias

If we assume $x_i = (1, x_{i,1}, \ldots, x_{i,d})^T$ (as in linear regression) we can also leave the b away

$$\hat{f}(x) = w^T x$$

Also note that in 2D the discriminant is a line, in 3D it is a plane, etc..

22.1.2 The perceptron classifier

Given the training data $\{(x_i, y_i)\}$ with $x_i \in \mathbb{R}^d$ and $y_i \in \{-1, 1\}$

How to find a weight vector w so that the separating hyperplane correctly separates the two categories?

1. Initialize $w = \mathbf{0}$
2. While there is an i such that $\hat{f}(x_i)y_i < 0$ do

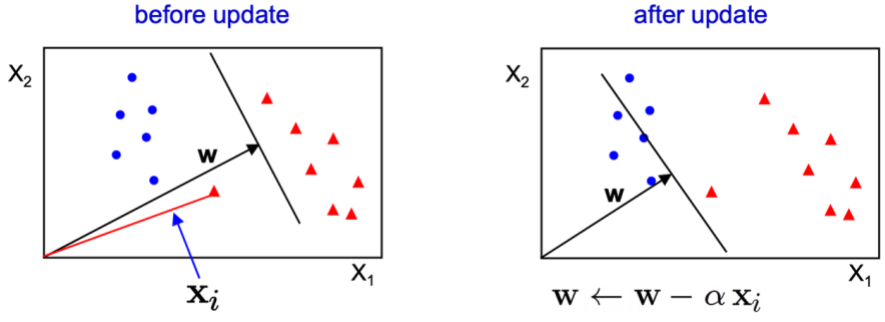
$$w := w - \alpha x_i \text{sign}\left(\hat{f}(x_i)\right)$$

With $\alpha > 0$ being a learning rate

At convergence

$$w = \sum_{i=1}^{N} \alpha_i x_i$$

With α_i being the accumulated weight for training sample x_i

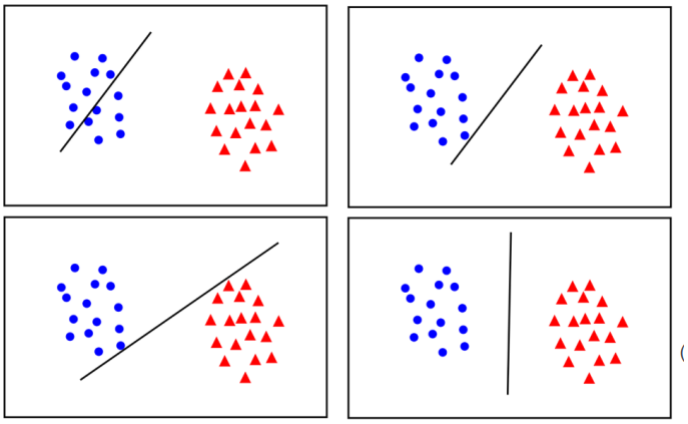


Convergence can be slow, but if there is a solution, the algorithm will find it. Separating line is also usually very close to training data

22.2 Support Vectors

What is the best w?

The idea is that we want the maximum margin solution, so that it is the most stable under perturbations.



We want bottom right

The support vectors are the values that are closest to the decision boundary. We want to equalize/maximize the distance from decision vectors to decision boundary.

22.2.1 SVM Optimization Problem

Distance $\delta(x_i, h)$ of data point x_i to hyperplane h

$$\delta(x_i, h) = \frac{y_i \cdot (w^T x_i + b)}{\|w\|}$$

In general, the length of vector w does not matter, we usually fix w so that the support vectors x_j make $y_j(w^T x_j + b) = 1$

The positive and negative support vectors have distance

$$\delta(x_j, h) = \frac{1}{\|w\|}$$

We want to maximize this.

The standard formulation is instead, we want to minimize $\frac{\|w\|}{2}$

$$\max_w \frac{2}{\|w\|}$$

but subject to

$$y_i(w^T x_i + b) \geq 1 \quad \text{for } i = 1, \ldots, N$$

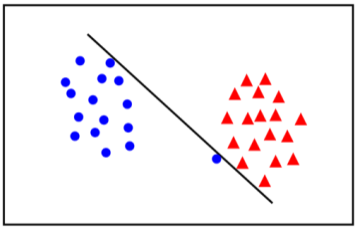
Or Equivalently

$$\min_w \|w\|^2$$

subject to the same constraints

This is a **quadratic optimization problem**

22.3 Soft margin and Hinge Loss



A single outlier completely destroys the optimization problem. Possible, the large margin solution is better, even though one constraint is violated.

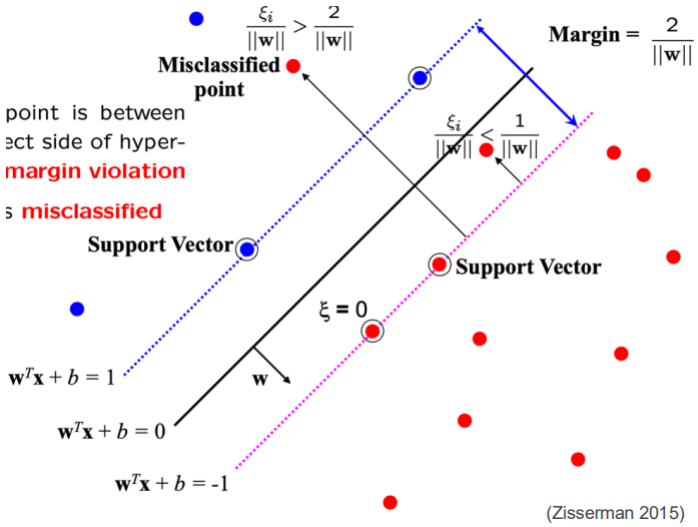
There is a trade-off between margin and number of mistakes

22.3.1 Introducing 'slack' variables

$\xi_i \geq 0$

The idea is that

- $0 < \xi \leq 1$, the point is between margin and correct side of hyperplane. **Margin violation**
- $\xi > 1$. The point is **misclassified**



Note the formulas for the margins, we want to maximize those margins

22.3.2 Soft margin solution

$$\min_{w \in \mathbb{R}^d, \xi_i \in \mathbb{R}^+} \|w\|^2 + C \sum_{i=1}^{N} \xi_i$$

subject to

$$y_i(w^T x_i + b) \geq 1 - \xi_i \quad \text{for } i = 1, \ldots, N$$

- Every constraint can be satisfied if our ξ_i is large enough.
- C is a **regularization parameter**. Small C allows constraints to be easily ignored (large margin), large C makes constraints hard to ignore (small margin), $C = \infty$ enforces all constraints

- Still quadratic optimization, but with a unique minimum.
- One hyperparameter C

22.3.3 Loss function

Given constraints

$$\xi_i \geq 0$$
$$y_i(w^T x_i + b) \geq 1 - \xi_i$$

And we can rewrite ξ_i as :

$$\xi_i = \max(0, 1 - y_i \hat{f}(x_i))$$

So we can optimize the unconstrained optimization problem over w

$$\min_{w \in \mathbb{R}^d} \underbrace{\frac{1}{C}\|w\|^2}_{\text{regularization}} + \sum_{i=1}^{N} \underbrace{\max(0, 1 - y_i \hat{f}(x_i))}_{\text{loss function}}$$

The points can be in three categories

1. $y_i \hat{f}(x_i) > 1$. Outside margin. No loss
2. $y_i \hat{f}(x_i) = 1$ On margin, no contribution to loss
3. $y_i \hat{f}(x_i) < 1$ Violates margin constraint, contributes to loss

SVM (Support Vector Machine) uses **Hinge loss** $\max(0, 1 - y\hat{f}(x_i))$

22.4 Gradient Descent over convex function

The idea is given a convex function, we want to climb 'down' the function, so go in the direction where

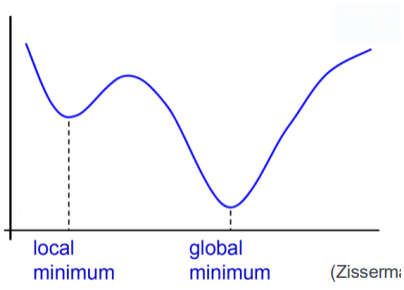
$$\frac{df(x)}{dx} = \nabla_x f(x)$$

is minimal

22.4.1 Optimization

Does this cost function have a unique solution?

$$\min_{w \in \mathbb{R}^d} C \sum_{i=1}^{N} \max(0, 1 - y_i \hat{f}(x_i)) + \|w\|^2$$

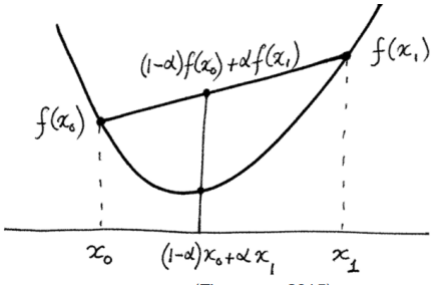


Can we find the solution through gradient descent?

22.4.2 Convex functions

A convex function $f : D \to \mathbb{R}$ is one that satisfies for any x_0, x_1 in D

$$f((1-\alpha)x_0 + \alpha x_1) \leq (1-\alpha)f(x_0) + \alpha f(x_1)$$



A non-negative sum of convex functions is convex

We can prove that the hinge loss and regularization (SVM) is a convex function.

22.4.3 Gradient Descent algorithm for SVM

To minimize a cost function $C(w)$ use the iterative update

$$w_{t+1} := w_t - \eta_t \nabla_w C(w_t)$$

where η is the learning rate

Let's rewrite the optimization problem as an average with $\lambda = \frac{2}{C}$

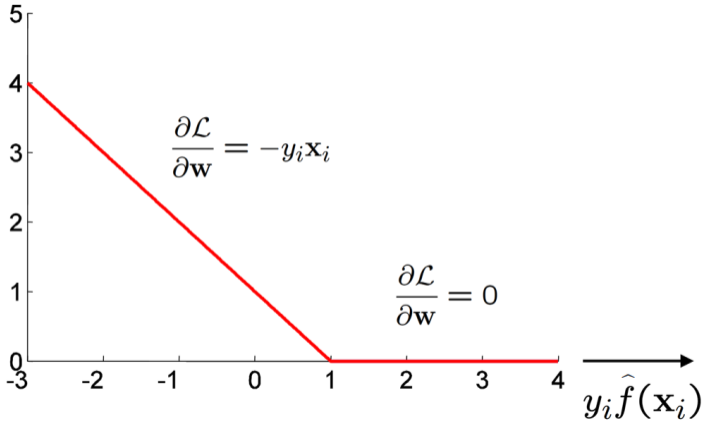
$$C(w) = \frac{1}{NC}\|w\|^2 + \frac{1}{N}\sum_{i=1}^{N} \max(0, 1 - y_i \hat{f}(x_i)) =$$

$$\frac{1}{N}\sum_{i=1}^{N}\left(\frac{\lambda}{2}\|w\|^2 + \max(0, 1 - y_i \hat{f}(x_i))\right)$$

and $\hat{f}(x_i) = w^T x + b$

Sub-gradient for Hinge loss :

$$L(x_i; y_i; w) = \max(0, 1 - y_i \hat{f}(x_i)), \hat{f}(x_i) = w^T x_i + b$$



Sub-gradient descent algorithm for SVM :

$$C(w) = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{\lambda}{2}\|w\|^2 + \mathcal{L}(x_i, y_i; w)\right)$$

The iterative update is

$$w_{t+1} := w_t - \eta\nabla_w C(w_t) :=$$

$$:= w_t - \eta\frac{1}{N}\sum_{i=1}^{N}(\lambda w_t + \nabla_w \mathcal{L}(x_i, y_i; w))$$

So then, each iteration t involves cycling through the training data with the updates

$$w_{t+1} := \begin{cases} w_t - \eta(\lambda w_t - y_i x_i) & \text{if } y_i\hat{f}(x_i) < 1 \\ w_t - \eta\lambda w_t & \text{otherwise} \end{cases}$$

Note: The typical learning rate is $\eta_t = \frac{1}{\lambda t}$

22.5 The dual problem

22.5.1 Primal vs dual problem

SVM is a linear classifier $\hat{f}(x) = w^T x + b$

The primal problem: an optimization problem over w

$$\min_{w\in\mathbb{R}^d} \frac{1}{C}\|w\|^2 + \sum_{i=1}^{N}\max(0, 1 - y_i\hat{f}(x_i))$$

The dual Problem: Getting rid of the w for a slightly different representation of $\hat{f}(x)$ that leads to the following representation

$$\hat{f}(x) = \sum_{i=1}^{N}\alpha_i y_i(x_i^T x) + b$$

and a new optimization problem with the same solution, but several advantages

22.5.2 Revisit optimization problem for hard margin case

Minimize the quadratic form

$$\frac{\|w\|^2}{2} = \frac{w^T w}{2}$$

with constraint

$$y_i(w^T x_i + b) \geq 1 \quad \forall i$$

The constraints will reach a value of 1 for at least one instance. If we include hard constraints into the loss function

$$\mathcal{L}(w, b, \alpha) = \frac{\|w\|^2}{2} - \sum_{i=1}^{N}\alpha_i(y_i(w^T x_i + b) - 1)$$

failed constraints punish the objective function

Lagrange multiplier (mental excursion) : We want to maximize function $f(x)$ under constraints $g(x) = a$. We can optimize with Lagrange multiplier $f(x) - \lambda(g(x) - a)$ instead

Algorithm for optimization with Lagrange Multiplier

1. Write down the Lagrangian $f(x) - \lambda(g(x) - a)$
2. Take derivative w.r.t x, set to 0. Find estimate of x that depends on λ
3. Plug estimate into Lagrangian, take derivative w.r.t λ, set to zero and find optimal value for lagrange multiplier λ
4. Plug in the lagrange multiplier in your estimate for x

And if we remember from above

$$\mathcal{L}(w, b, \alpha) = \frac{\|w\|^2}{2} - \sum_{i=1}^{N} \alpha_i (y_i (w^T x_i + b) - 1)$$

α_i are the Lagrange multipliers

Lagrangian primal problem

$$\min_{w,b} \max_{\alpha} \mathcal{L}(w, b, \alpha)$$

under constraint $\forall i : \alpha_i \geq 0$

Finding the optimum : Loss is a function of w, b, α

We find the optimum using derivatives

$$\frac{\partial}{\partial b} \mathcal{L}(w, b, \alpha) = 0 \Rightarrow 0 = \sum_{i=1}^{N} \alpha_i y_i$$

$$\frac{\partial}{\partial w_j} \mathcal{L}(w, b, \alpha) = 0 \Rightarrow w_j = \sum_{i=1}^{N} \alpha_i y_i x_{i,j} \Rightarrow w = \sum_{i=1}^{N} \alpha_i y_i x_i$$

Substitution (not done by hand)

$$\mathcal{L}(w, b, \alpha)_{|w=\sum_{i=1}^{N} \alpha_i y_i x_i} = \frac{\|w\|^2}{2} - \sum_{i=1}^{N} \alpha_i (y_i \cdot (w^T x_i + b) - 1) =$$

$$= \frac{1}{2} \left\{ \sum_{j=1}^{N} \alpha_j y_j x_j \right\}^T \left\{ \sum_{k=1}^{N} \alpha_k y_k x_k \right\} - \sum_{i=1}^{N} \alpha_i \left(y_i \cdot \left(\left\{ \sum_{j=1}^{N} \alpha_j y_j x_j \right\}^T x_i + b \right) - 1 \right) =$$

$$= \underbrace{\frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (x_j^T x_k)}_{=} - \underbrace{\sum_{i,j} \alpha_i \alpha_j y_i y_j (x_i^T x_j)}_{=} - b \underbrace{\sum_{i=1}^{N} \alpha_i y_i}_{=0} + \sum_{i=1}^{N} \alpha_i =$$

$$= \mathcal{L}(\alpha) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (x_j^T x_k)$$

22.06.2026 53

22.5.3 Wolfe-dual Problem

$$\max_{\alpha} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (x_j^T x_k)$$

subject to $\forall i : \alpha_i \geq 0$ and $0 = \sum_{i=1}^{N} \alpha_i y_i$

This problem is solvable with quadratic programming. It gives us the classification function

$$\hat{f}(x) = \sum_{i=1}^{N} \alpha_i \cdot y_i \cdot x_i^T x + b$$

22.5.4 Summary: Primal and dual formulations

Primal version of the classifier

$$\hat{f}(x) = w^T x + b$$

Dual version of the classifier

$$\hat{f}(x) = \sum_{i=1}^{N} \alpha_i y_i x_i^T x + b$$

The dual form seems to work like a kNN classifier, because it requires training points x_i, but in practice many of the α_i are zero. The ones that are non-zero define the support vectors x_i

The Lagrangian primal problem

$$\min_{w,b} \max_{\alpha} \mathcal{L}(w, b, \alpha)$$

subject to $\forall i : \alpha_i \geq 0$

The Lagrangian dual problem

$$\max_{\alpha} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (x_j^T x_k)$$

Subject to $\forall i : \alpha_i \geq 0$ and $0 = \sum_{i=1}^{N} \alpha_i y_i$

22.6 Kernelization Tricks in SVMs

The thing is that not all classes can be separated via a hyperplane. The solution is **feature engineering**

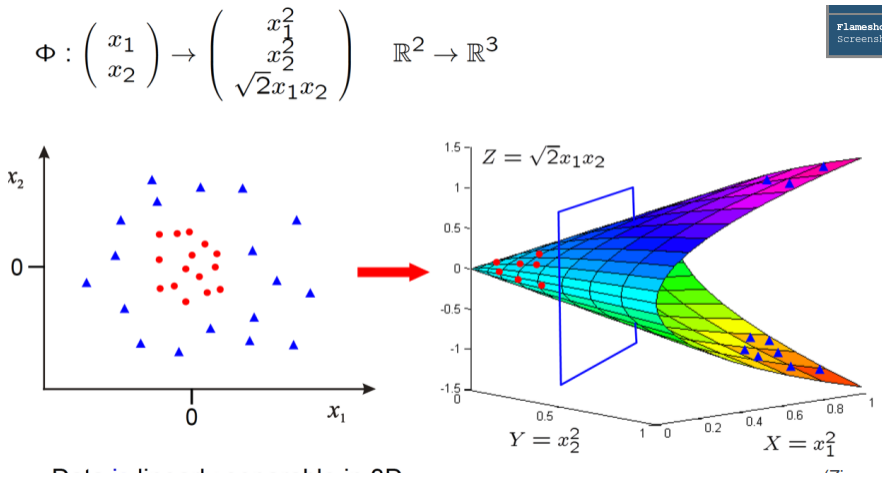
22.6.1 Feature engineering using $\phi(x)$

Classifier: Given $x_i \in \mathbb{R}^d, \phi : \mathbb{R}^d \to \mathbb{R}^D$, $w \in \mathbb{R}^D$

$$\hat{f}(x) = w^T \phi(x) + b$$

Learning

$$\min_{w \in \mathbb{R}^D} \frac{1}{C} \|w\|^2 + \sum_{i=1}^{N} \max(0, 1 - y_i \hat{f}(x_i))$$



The effect of applying functions when feature engineering, allows us to generate a valid hyperplane.

The idea is usually mapping to a higher dimensional space D where the data can be separated with a hyperplane.

However, if $D \gg d$ there are many more parameters to learn for w

22.6.2 Dual classifier in transformed feature space

Classifier

$$\hat{f}(x) = \sum_{i=1}^{N} \alpha_i y_i x_i^T x + b \Rightarrow$$

$$\Rightarrow \hat{f}(x) = \sum_{i=1}^{N} \alpha_i y_i \phi(x_i)^T \phi(x) + b$$

And for learning its analogous

$$\max_{\alpha} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (\phi(x_j)^T \phi(x_k))$$

Subject to the usual.

Kernels

$$k(x_j, x_k) = \phi(x_j)^T \phi(x_k)$$

22.6.3 Dual classifier using kernels

The same as above only with the kernel substitution

Classifier

$$\hat{f}(x) = \sum_{i=1}^{N} \alpha_i y_i k(x_i, x) + b$$

Learning

$$\max_{\alpha} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k k(x_j, x_k)$$

22.7 Kernels

Formal definition: Given mapping $\phi : \mathcal{X} \to \mathcal{H}$ from input space $\mathcal{X}$ to a possibly infinitely-dimensional space $\mathcal{H}$. A kernel function k is defined as

$$k(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}$$

It computes the inner product between the images of x and x', but without needing to know ϕ explicitly.

22.7.1 Properties

- Symmetric
- Positive semi-definite.

22.7.2 Examples

- Linear kernels:
$$k(x, x') = x^T x'$$
- Polynomial kernels:
$$k(x, x') = (1 + x^T x')^d$$
- Gaussian kernels:
$$k(x, x') = e^{-\frac{\|x-x'\|^2}{2\sigma^2}} \quad \sigma > 0$$
Infinite dimensional feature space. Also called Radial basis function kernel (RBF)

22.8 Transductive Classification

They are special SVM's that use test data, not only training data with the Objective function

$$\min_{w,b,\xi,\xi^*} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{m} \xi_i + C^* \sum_{j=1}^{k} \xi_j^*$$

- $\frac{1}{2}\|w\|^2$: Standard SVM objective to maximize the margin ($\frac{2}{\|w\|}$).
- $C\sum_{i=1}^{m} \xi_i$: Penalty for misclassifications or margin violations on the labeled training data (m samples).
- $C^* \sum_{j=1}^{k} \xi_j^*$: Penalty for margin violations on the unlabeled test data (k samples).
- C and C^*: Hyperparameters balancing the trade-off between margin width, training error, and test set assignment confidence.

Subject to

$$y_i(w^T x_i + b) \geq 1 - \xi_i \quad \forall i = 1, \ldots, m$$
$$y_j^*(w^T x_j^* + b) \geq 1 - \xi_j^* \quad \forall j = 1, \ldots, k$$
$$\xi_i \geq 0, \quad \xi_j^* \geq 0$$

23 Neural Networks

23.1 Biological vs Artificial

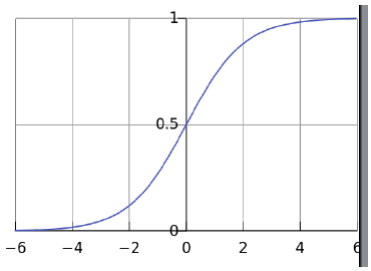
The workings on natural neural networks motivated research on artificial neural networks.

23.2 Reminder: Logistic Regression

Logistic regression is a machine learning method where you predict probabilities of an event ocurring. Mainly used for binary classification tasks.

You have a **model**, composed of

- A linear function $z = x^T\beta$
- A non-linear activation function
$$y = \frac{e^z}{1+e^z}$$
- Together they form
$$\hat{f}(x) = \frac{e^{x^T\beta}}{1+e^{x^T\beta}}$$



And you also have a loss function, based on **cross entropy** and **regularization**

$$\text{loss}^{\text{logistic}}(\beta) = \sum_{i=1}^{N} H(\overline{y}_i, P(\cdot|X = x_i)) + \lambda||\beta||^2$$

Note: (the $\lambda||\beta^2||$ is the L2 Regularization that penalizes large weights)

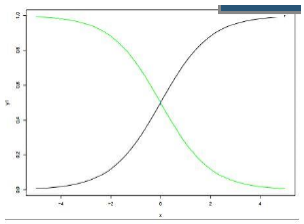
To solve, you adapt the weights, based on something like **gradient descent**

23.2.1 Logistic regression for binary cases

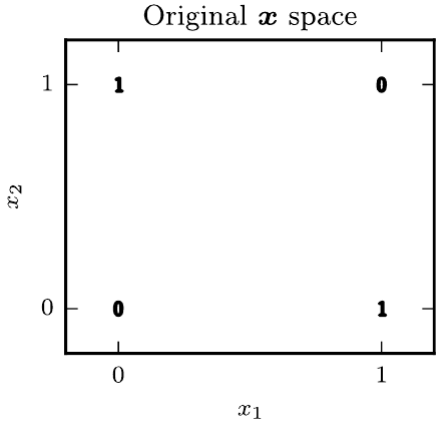
In a two class example, you want yo predict conditional probabilities. (Two class example)

$$P(G = 1|X = x) = \frac{e^{x^T\beta}}{1+e^{x^T\beta}}$$

$$P(G = 2|X = x) = \frac{1}{1+e^{x^T\beta}}$$



Problem: Learning XOR with logistic regression : So XOR is a diagonal problem, so it cannot be cleanly divided with a single linear classifier (which the logistic regression is).



This is because logistic regression can only draw a hyperplane (a line in 2D, and a plane in 3D), so it cannot inherently learn the XOR problem.

To fix this, there are two main methods

- Kernelization: Modifying the data so that it is separable
- Composition of non-linear functions (which is the basis of Neural Networks)

23.3 Feed-Forward Networks and Activation Functions

23.3.1 Network: Composing Functions

A three layer network

$$f(x) = f^{(3)}(f^{(2)}(f^{(1)}(x)))$$

With the last layer $f^{(3)} = \hat{y}$ being the output layer.

Composition of linear functions for XOR Problem? If we have layer 1 (hidden layer)

$$h = f_1(x) = W^T x + c$$

And layer 2 (output layer)

$$y = f_2(h) = w^T h + b$$

Then $f(x) = w^T W^T x$, and the result is still linear. So combinations of linear functions still yields linear functions, so we cannot use it for the XOR problem.

23.3.2 One Layer in a Feed-Forward Network

The output of a single unit (neuron) within an arbitrary layer i is defined by a linear combination of non-linear features:

$$f^{(i)}(x) = w_{i,0} + \sum_{j=1}^{l_i} w_{i,j} \cdot \phi(x; \theta_i) = \phi(x; \theta_i)^T w_i$$

With:

- l_i being the number of incoming features (the width/dimension of the preceding layer).
- $w_{i,j}$ being the weight in layer i associated with the j-th incoming feature, and $w_{i,0}$ being the bias term.

- $\phi(x;\theta_i)$ being a basis function used to extract features. Instead of hand-crafting this function, deep learning makes it learnable by nesting a parameterized linear component inside a fixed, non-linear activation function $g(z)$:

$$\phi(x;\theta_i) = g(x^T\theta_i)$$

  where θ_i represents the weights of the previous layer that generate these features.

Activation functions : This has been a long debate on which to use, below are the most common

- Sigmoid: $g(z) = \sigma(z) = \frac{e^z}{1+e^z}$
- ReLU: $g(z) = \max\{0, z\}$
- Leaky ReLU: $g_\alpha(z) = \max\{\alpha z, z\}$
- Maxout unit: $g(z) = \max_j\{z_j\}$
- Absolute value: $g(z) = |z|$
- Hyperbolic tangent: $g(z) = \tanh(z)$
- Radial Basis function: $g(z) = e^{-\frac{1}{2}||z_j - c_j||^2}$

The most used is ReLU because of speed and simplicity, but it lacks differentiability at 0.

Using this method, we CAN solve the XOR problem

23.3.3 Universal Approximation Theorem

Let $\varphi : \mathbb{R} \to \mathbb{R}$ be a nonconstant, bounded, and monotonically-increasing continuous function.

Then let $\mathcal{C}$ denote the space of continuous functions on $[0,1]^m$

Given **any** $\varepsilon > 0$ and **any** function $f \in \mathcal{C}$

there exist, $N \in \mathbb{N}$, $v_i, b_i \in \mathbb{R}$, $w_i \in \mathbb{R}^m$, with $i = 1, ..., N$ where

$$F(x) := \sum_{i=1}^{N} v_i \varphi(w_i^T x + b_i)$$

and

$$|F(x) - f(x)| < \varepsilon \quad \forall x \in [0,1]^m$$

So translated, on compact spaces, sufficiently broad neural networks can approximate any continuous function arbitrarily well. However, this is not practical, as there is no guarantee that N is even non-exponential.

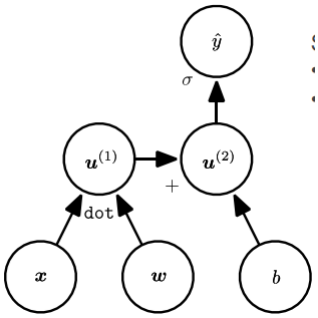
As a summary, we are still missing a Loss function and the solving for One-Layer Feed Forward Networks.

23.4 Backpropagation

It is used to compute the Loss $L(\theta)$ considering data y and $\hat{y}$ (output prediction)

23.4.1 Computation Graphs: Language for representing the backpropagation problem

It's a language to represent mathematical expressions using graphs. They are pretty self explanatory.



Notational Conventions There are n many nodes in the graph, topologically sorted. $u_1, ..., u_n$
The input layers with values for k nodes are $u_1, ..., u_k$.
The set of values of parent of node i is $\mathbb{A}^{(i)}$
Operator for node i is $f^{(i)}$
Output layer with l nodes $u_{n-l+1}, ..., u_n$
So in the above example the distribution would be

- $u_1 = x_1$
- $u_2 = w$
- $u_3 = b$
- $u_4 = u^{(1)}$
- $u_5 = u^{(2)}$
- $\mathbb{A}^{(4)} = \{u_1, u_2\} = \{x, w\}$
- $\mathbb{A}^{(5)} = \{u_3, u_4\} = \{b, u^{(1)}\}$
- $\mathbb{A}^{(6)} = \{u_5\}$
- $f^{(4)} = dot$
- $f^{(5)} = +$
- $f^{(6)} = \sigma$
- $u_6 = \hat{y}$

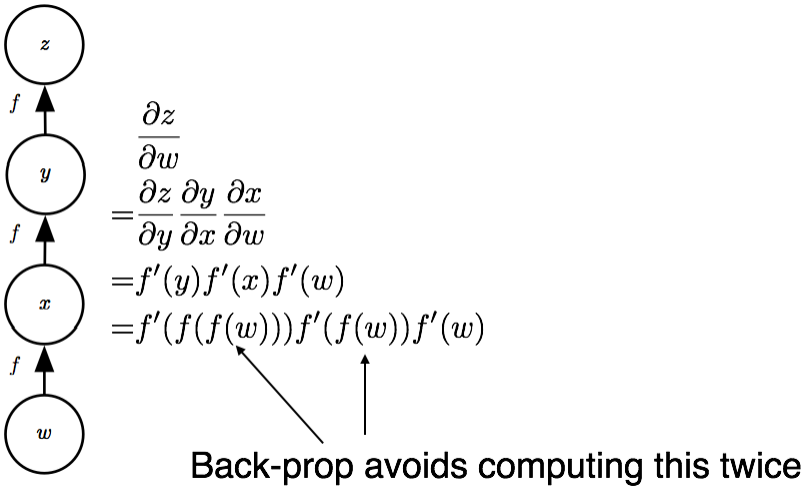
Forward propagation in computational graph :

---

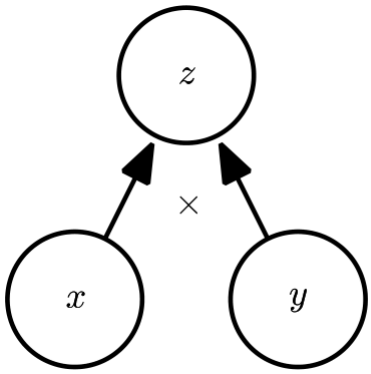
for i = k+1,...,n do:
  $\mathbb{A}^{(i)} \leftarrow \{u_j | j \in parent(u_i)\}$
  $u_i \leftarrow f^{(i)}(\mathbb{A}^{(i)})$
end for
return u_n

23.4.2 Backpropagation

Below is an easy example



Example in a tiny computational graph :



Imagine we have input $x = 1$, $y = 2$ and target $z = 3$. The forward result is $\hat{z} = x \cdot y = 2$

If we assume MSE as our loss function

$$J = L(\hat{z}, z) = \frac{1}{2}(\hat{z} - z)^2 = \frac{1}{2}$$

We want to now compute the effect of $\hat{z}$ on the loss function. (We use here partial and not gradient because $\hat{z}$ is a scalar and not a vector)

$$\frac{\partial J}{\partial \hat{z}} = \frac{\partial}{\partial \hat{z}}\left[\frac{1}{2}(\hat{z} - z)^2\right] = (\hat{z} - z)$$

When plugging in our numbers we get

$$\frac{\partial J}{\partial \hat{z}} = (2 - 3) = -1$$

This means the gradient direction is negative. Now via the chain rule we can get the effect of x and y

$$\nabla_{(x,y)} J = \begin{pmatrix} \frac{\partial J}{\partial x} \\ \frac{\partial J}{\partial y} \end{pmatrix} = \begin{pmatrix} \frac{\partial J}{\partial \hat{z}} \frac{\partial \hat{z}}{\partial x} \\ \frac{\partial J}{\partial \hat{z}} \frac{\partial \hat{z}}{\partial y} \end{pmatrix} = \begin{pmatrix} -2 \\ -1 \end{pmatrix}$$

Since derivative measures change, a negative gradient means that increasing x or y will decrease the loss J.

So for example for y

$$y^{new} = y - \eta \nabla_y = y - \eta(-1)$$

With learning rate ∇

23.5 Modelling a Feed-Forward Network predicting a Bernoulli distribution

Note: A Bernoulli distribution only has two outcomes: 1 with probability p and 0 with probability $1 - p$.

23.5.1 Loss function with Maximum Likelihood

Given training data $\{(x_i, y_i)\}_{i=1}^N$ where $x_i \in \mathbb{R}^d$ and $y_i \in \{0, 1\}$.

The cross-entropy loss over the dataset is defined as:

$$L(\theta) = \mathbb{E}_{\text{data}}(-\log P(y|x)) = -\frac{1}{N} \sum_{i=1}^{N} \log P(y_i|x_i)$$

23.5.2 Probabilistic Model and Activation

Let the linear logit layer be defined as $z = w^T x + b$. Intuitively, we want z to be a large positive number when the true label $y = 1$, and a large negative number when $y = 0$. This relationship is compactly expressed by modeling the unnormalized conditional log-probability as:

$$\log P(y|x) \propto (2y - 1)z \Rightarrow P(y \mid x) \propto e^{(2y-1)z}$$

Note that these scores are unnormalized (meaning they don't sum up to one). This is fixed below

To handle both the $y = 1$ and $y = 0$ cases simultaneously with a single elegant expression, we map the binary target $y \in \{0, 1\}$ to the sign domain $y' = 2y - 1 \in \{-1, 1\}$:

- If $y = 1 \implies y' = 1$ (demanding a large positive z)
- If $y = 0 \implies y' = -1$ (demanding a large negative z)

Normalizing these exponential probabilities (by dividing with all the possible outputs) so that $P(y = 1|x) + P(y = 0|x) = 1$ yields:

$$P(y|x) = \frac{e^{yz}}{\sum_{y'=0}^{1} e^{y'z}} = \frac{e^{(2y-1)z}}{1 + e^{(2y-1)z}} = \sigma((2y - 1)z)$$

where $\sigma(a) = \frac{1}{1+e^{-a}}$ is the standard sigmoid activation function.

23.5.3 Negative Log-Likelihood Formulation

Taking the negative log of the normalized probability for a single sample gives:

$$L_i(\theta) = -\log P(y|x) = -\log \sigma((2y - 1)z)$$

Using the identity $-\log \sigma(a) = \zeta(-a)$, where $\zeta(a) = \log(1 + e^a)$ is the softplus function, the loss simplifies cleanly to:

$$L_i(\theta) = \zeta((1 - 2y)z)$$

23.5.4 Softplus function ζ

$$\zeta(x) = \log(1 + e^x)$$
$$\frac{d\zeta}{dx}(x) = \sigma(x)$$
$$\log(\sigma(x)) = -\zeta(-x)$$

23.5.5 Summary: Learning Bernoulli Output

The model is $\hat{f}(x) = \sigma(w^T x)$

The loss function is **softplus**

$$L(\theta) = \zeta((1 - 2y)z)$$

And we solve with gradient descent

$$\frac{d\zeta(x)}{dx} = \sigma(x)$$

23.5.6 Softmax vs Softplus

- Softplus operates on a single scalar input to ensure the output is strictly positive (> 0).
- Softmax operates on a vector of inputs to convert them into a normalized probability distribution ($\sum = 1$).

23.6 Modelling a Feed-Forward Network predicting a Multinoulli distribution

In Bernoulli it was

$$\hat{y} = P(y = 1|x)$$

With Multinoulli

$$\hat{y}_i = P(y = i|x)$$

With $\hat{y} = (\hat{y}_1, ..., \hat{y}_n)$ being the vector of predicted values from our Neural Network for n different classes. With the property of

$$\forall i \quad \hat{y}_i \in [0, 1]$$

and

$$\sum_{i=1}^{n} \hat{y}_i = 1$$

Basically, a Bernoulli distribution with more than 2 outcomes, like rolling a 6-sided die.

So before, z and b were scalars and w and h were vectors, now z and b and h (can also be called x, depends if its true input or previous layer) are vectors and w is a matrix.

We also use the **softmax function**

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

The issue is that the denominator can overflow when computing inside a computer. So we pick $c := \max_i z_i$ and just do

$$\text{softmax}(z)_i = \frac{e^{z_i - c}}{\sum_j e^{z_j - c}}$$

This is called **stable softmax**

23.6.1 Multinoulli Loss Function (Cross-Entropy)

For a single data record, let the true class label be represented as a one-hot encoded vector $y = [0, \ldots, 0, 1_i, 0, \ldots, 0]^T$ (often written simply as $y = i$, where i is the correct class index).

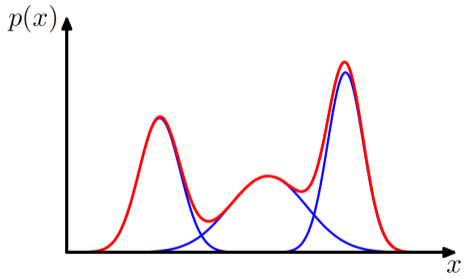
The cross-entropy loss measures the distance between the true data distribution and our model's learned distribution:

$$\begin{aligned} L_i(\theta) &= -\sum_j y_j \log(\text{softmax}(z)_j) \\ &= -\log(\text{softmax}(z)_i) \\ &= -\log\left(\frac{e^{z_i}}{\sum_j e^{z_j}}\right) \\ &= -\left(\log e^{z_i} - \log\sum_j e^{z_j}\right) \\ &= \log\sum_j e^{z_j} - z_i \end{aligned}$$

This final form shows that to minimize the loss, the network must pull the correct logit z_i up, while pushing the sum of all exponentials $\sum_j e^{z_j}$ down.

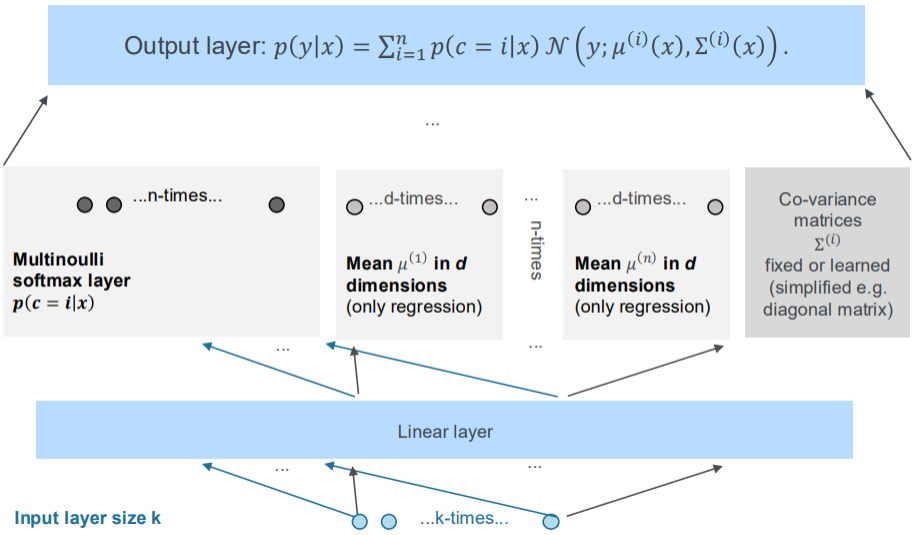
23.7 Modeling a Mixture Density distribution

Imagine we want to model a distribution of dogs, two breeds: Chihuahuas and Siberian Huskys. They vary wildly in weight, so the final distribution will be a **mixture of gaussians**.



This is the regression result of regression of a Gaussian Mixture. Luckily this is univariate, for multivariate it turns even harder, with multiple dimensions.

The approach mainly to solve this is to use a Multinoulli softmax layer to try to categorize the groups, and then computing the means, and then the covariance matrices



The Loss function used is usually the **Mean Squared Error**, with some variations.
The prediction is then

$$\mathbb{E}[y|x] = \int yP(y|x)dy = \sum_{i=1}^{N} P(c = i)\mu^{(i)}(x)$$

23.8 Optimizing Gradient Descent

23.8.1 Initialization

- The initialization of weights is important, there are some heuristics
- Choose random weights that don't grow or vanish the gradients. Choose each weight vector $W_{l,i\cdot}$ to point in a uniform random direction.
- Choose biases $b_{l,i}$ randomly so that ReLU hinges over the input well

23.8.2 Optimization

- For small data size: Compute the loss and it's gradient

$$\sum_{i=1}^{n} \nabla_\beta l(f_\beta(x_i), y_i)$$

  Then use classic gradient-based optimization methods

- For large data size the $\sum_{i=1}^{N}$ is largely inefficient! So we adapt weights based on much smaller data subsets, **mini batches**

Newton's method in optimization (1D) : To minimize $f(x)$, iterate $t = 0, 1...$ for x_t to approach minimum

Taylor expansion

$$f(x_t + \varepsilon) \approx f(x_t) + f'(x_t)\varepsilon + \frac{1}{2}f''(x_t)\varepsilon^2$$

If we derive and set to zero

$$\varepsilon = -\frac{f'(x_t)}{f''(x_t)}$$

So then to optimize, we just set $x_{t+1} = x_t + \varepsilon$

And to do in several dimensions: We call the gradient

$$g_f(x) = \nabla f(x) = \left(..., \frac{\partial f(x_1, ..., x_i, ...)}{\partial x_i}, ... \right)^T$$

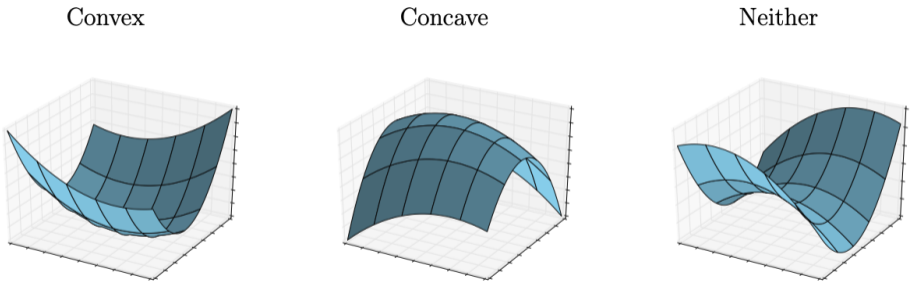
The Hessian is then

$$H_f(x) = \nabla^2 f(x) = \begin{pmatrix} ... & ... & ... \\ ... & \frac{\partial^2 f(x_1,...,x_i,...)}{\partial x_i^2} & .... \\ ... & ... & ... \end{pmatrix}$$

So then $x_{t+1} = x_t - \eta[H_f(x_t)]^{-1} g_f(x_t)$

With $\eta < 1$ we dampen the search.

Illustrating and Computing the Hessian : The Hessian is symmetric, and largest and smallest eigenvector illustrate the largest and smallest curvature.



To compute, inverting the Hessian is expensive and should be avoided. So we do not compute $h(x_t) = [H_f(x_t)]^{-1} g_f(x_t)$. Rather we solve the linear equation

$$H_f(x_t) h(x_t) = g_f(x_t)$$

Convergence : For strongly convex functions with Lipschitz Hessian matrix, meaning

$$||\nabla^2 f(x) - \nabla^2 f(y)||_{op} \leq L||x - y||_2$$

we have quadratic convergence.

Convexity is generally not given for neural networks, but for other ML methods (SVM).

Summary: Convergence is not typically guaranteed for NN, but practically, the high dimensionality usually allows for avoiding many local minima

23.8.3 Stochastic Gradient Descent

So we instead compute the loss and gradient for a mini-batch $\hat{D} \subset D$ of fixed size k

$$L(\beta, \hat{D}) = \sum_{i \in \hat{D}} l(f_\beta(x_i), y_i)$$

And the gradient

$$\nabla_\beta L(\beta, \hat{D}) = \sum_{i \in \hat{D}} \nabla_\beta l(f_\beta(x_i), y_i)$$

Naive Stochastic Gradient Descent : We iterate

$$\beta \leftarrow \beta - \eta \nabla_\beta L(\beta, \hat{D})$$

The choice of η is key for convergence. We can use exponential cooling $\eta = \eta_0^t$

Adding Momentum : SGD with momentum

$$\Delta\beta \leftarrow \alpha\Delta\beta - \eta \nabla_\beta L(\beta, \hat{D})$$

$$\beta \leftarrow \beta + \Delta\beta$$

And with **Nesterov Accelerated Gradient (NAG)**

$$\Delta\beta \leftarrow \alpha\Delta\beta - \eta \nabla_\beta L(\beta + \Delta\beta, \hat{D})$$

$$\beta \leftarrow \beta + \Delta\beta$$

23.9 ADAM - Adaptive Moment Estimation

Will not appear in exam.

23.10 Regularization in DNNs: Lasso, Ridge and Constraints

Regularization is defined as any modification done to a learning algorithm intended to reduce its generalization error, but not its training error.

DNN = Deep Neural Network

23.10.1 Paramer Norm Penalty

Regularized loss function

$$\tilde{J}(\theta; X, y) = \underbrace{J(\theta; X, y)}_{\text{Unregularized}} + \alpha \underbrace{\Omega(\theta)}_{\text{Regularizer on parameters}}$$

Note: w only denotes the weights, while θ refers to **all parameters**

23.10.2 L^2 Parameter Regularization

$$\tilde{J}(w; X, y) = J(w; X, y) + \frac{\alpha}{2} w^T w$$

The bias vectors are usually not regularized

So the regularized gradient

$$\nabla_w \tilde{J}(w; X, y) = \nabla_w J(w; X, y) + \alpha w$$

If we use the regularized gradient for the weight updating.

$$w \leftarrow w - \varepsilon(\alpha w + \nabla_w J(w; X, y))$$

23.10.3 L_1 and L_2 regularization by weight decay

Add penalty for every weight (Ridge or Lasso), and in practice, L_1 makes the weights become sparser than L_2 (L_1 pushes the weights toward 0 more often)

23.10.4 Column-Wise (Matrix) Regularization

In multi-layer models or multi-output regression, the weights for layer l form a 2D matrix $W_l \in \mathbb{R}^{n \times m}$:

$$W_l = \begin{pmatrix} w_{l,1,1} & \dots & w_{l,1,m} \\ \vdots & \ddots & \vdots \\ w_{l,n,1} & \dots & w_{l,n,m} \end{pmatrix}$$

where n is the number of output units, m is the number of input units, and $w_{l,i,j}$ represents the weight connecting input j to output i.

To apply L_2 regularization (Ridge) to a weight matrix, we use the **Frobenius norm** $\|W_l\|_F$, which sums the squared magnitude of every individual weight in the matrix:

$$\|W_l\|_F^2 = \sum_{i=1}^{n} \sum_{j=1}^{m} |w_{l,i,j}|^2$$

23.10.5 Regularization in DNNs: Data Augmentation

You can generate more data by taking the data you have and modifying it slightly. Rotate it, add some noise, flip it. You basically get more data for free. But be careful (a rotated 6 is a 9).

Label Smoothing: Injecting noise at output target Most datasets have some number of mistakes in the y labels. It can be harmful to maximize $P(y|x)$ if y is a mistake.

The solution can be model the noise at the label

$$\frac{\varepsilon}{k-1}, ..., 1-\varepsilon, \frac{\varepsilon}{k-1}, ...$$

Injecting noise into the weights : We can wiggle all the weights with some noise ε_W

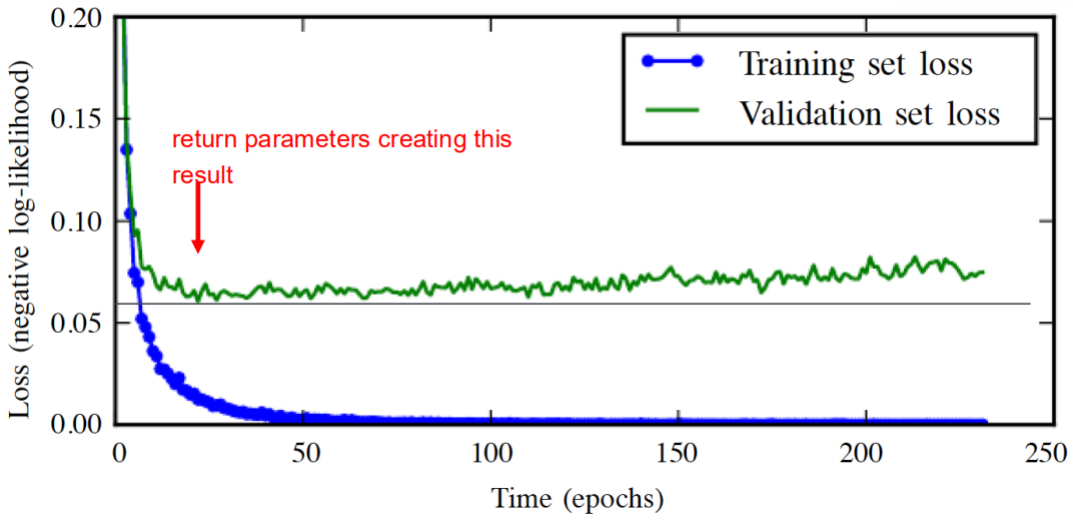
We propagate with wiggle weights to compute $\hat{y}_{\varepsilon_W}$

This aims to avoid getting stuck at local minima

23.10.6 Multi-task learning

From one data point x several outputs $y^{(i)}$ representing different tasks

23.10.7 Early stopping



As you can see, to many epochs lead to overfitting, we want to return the optimal result for the validation set loss

23.10.8 Adversarial Examples

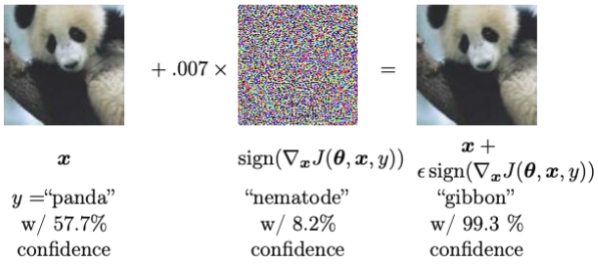
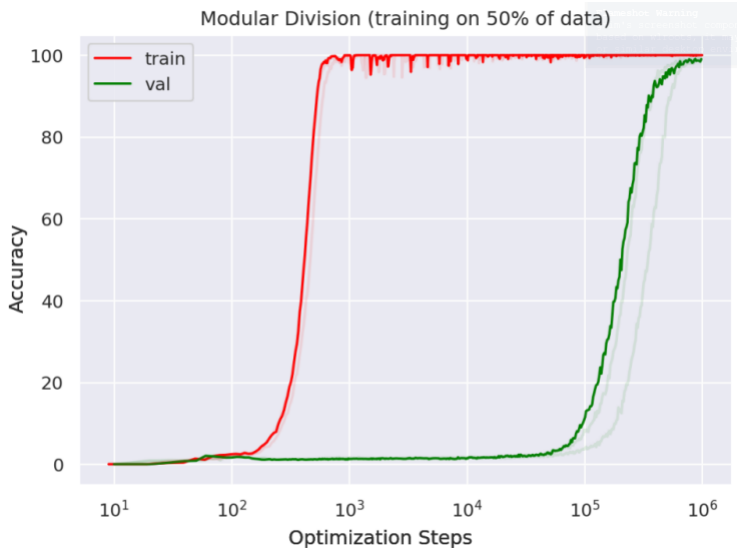


Figure 7.8

You can see that adding some noise can wildly change the image. Using this in training can improve security.

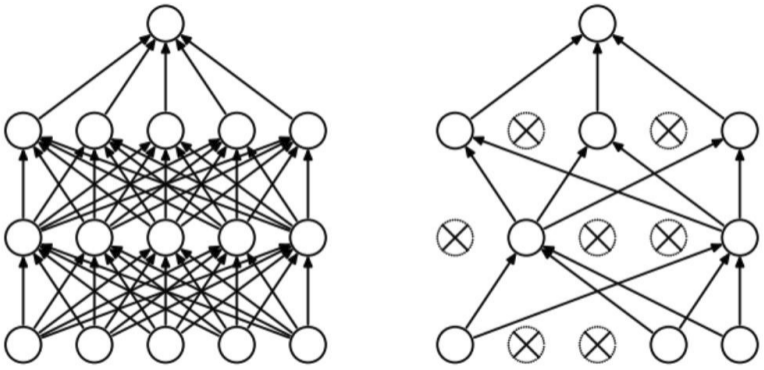
23.10.9 Grokking

Not yet peer reviewed, but is the phenomenon where training a long long time reduces the test error a lot



But then there is also **anti-grokking** where if you run it even longer, it ends up overfitting. It's a new subject and not too explored.

23.11 Regularization in DNNs: Dropout



(a) Standard Neural Net (b) After applying dropout.

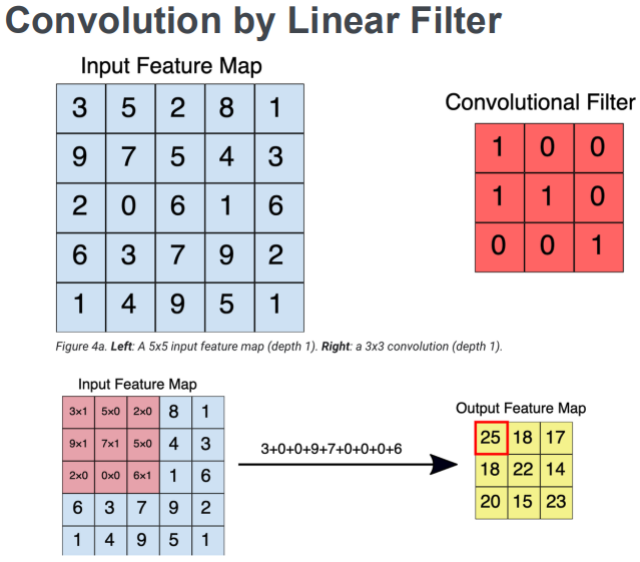
Per learning cycle, we stochastically eliminate neurons with probability p_l^{drop} in layer l. The intuition behind this is that we can't rely on any single feature, so we spread out the weights. It also avoids overfitting

He skipped a ton of slides here when doing the Lecture, I highly doubt it will come up in the exam.

23.12 Convolutional Neural Networks

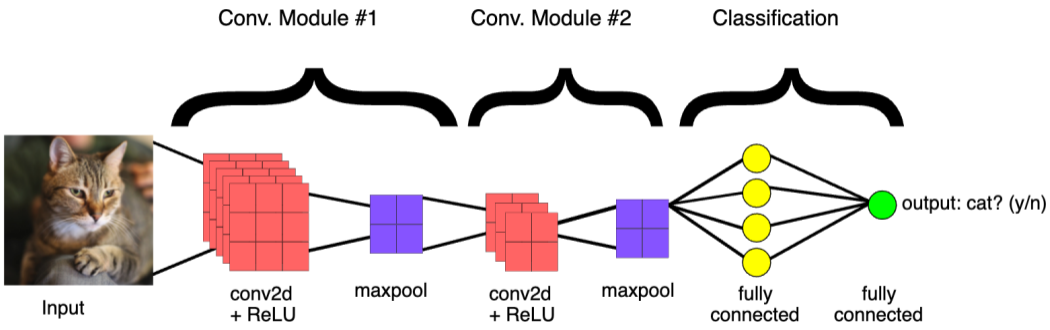
So far we had many parameters, and fully connected feed-forward networks. This is not suitable for every task. For images, why should an object be treated differently based on location for example?

So the idea for CNN is local vs global processing of image features.

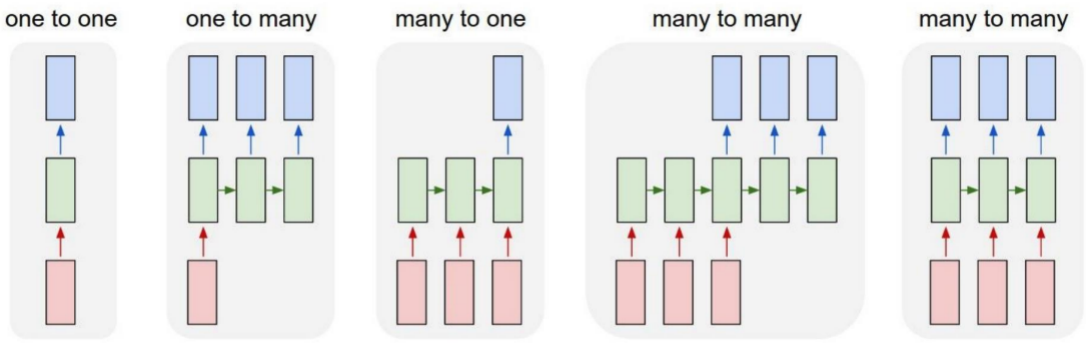


23.12.1 Reduce Dimensionality: Subsampling by pooling

The main method is called **maxpool**. We have a pool of size $n \times n$ and we just get the max value. Same as a convolution, but we just access the max value.



23.13 Recurrent Neural Networks



Our 'Vanilla' NN until now were the one to one model. The one-to-many could be image captioning (from image to sequence of words). The many-to-one could be Sentiment Classification (sequence of words to sentiment). Many-to-many could be machine translation (words to different words). And the last many-to-many could be video classification on frame level.

Recurrent Neural Networks are based on processing a vector x by applying a **recurrence formula** at every time step

$$\underbrace{h_t}_{\text{new state}} = \underbrace{f_W}_{\text{function with params W}}\left(\underbrace{h_{t-1}}_{\text{old state}}, \underbrace{x_t}_{\text{input vector at some time step}}\right)$$

So the feedback from step t to input at next time step $t+1$. Can be used for prediction of vehicle motion.

23.14 When to use DNNs

23.14.1 Curse of Dimensionality

- Low Dimensionality: x has few neighbors which are used to predict y
- High Dimensionality: x has (too) many neighbors in roughly the same distance, hard to predict y

This is true for kNN, Decision Trees, SVMs, Bayes, etc.

Some last comments regarding manifolds, if they come up I will get angry >: (

AND WE ARE DONE!!